



Unidad Didáctica 4: Introducción al Deep Learning y LSTM

Técnicas de AI Regresiones, deep learning y otras

ÍNDICE

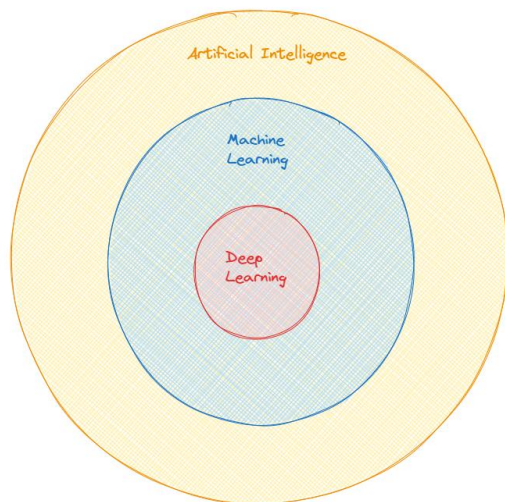
Contenido

1.	INTRODUCCIÓN.....	4
2.	OBJETIVOS.....	5
2.1	OBJETIVO GENERAL	5
3.	TEMA 1: INSPIRACIÓN BIOLÓGICA.....	6
4.	TEMA 2: QUÉ ES EL DEEP LEARNING.....	6
5.	TEMA 3: MATEMÁTICAS TRAS EL DEEP LEARNING	7
6.	TEMA 4: REDES NEURONALES.....	9
6.1.	NEURONA BIOLÓGICA	10
6.2.	PERCEPTRÓN	11
6.3.	MULTILAYER FEED-FORWARD NETWORKS	12
6.4.	BACKPROPAGATION	14
6.5.	FUNCIONES DE ACTIVACIÓN	14
6.6.	FUNCIÓN DE PÉRDIDAS	17
6.7.	HIPERPARÁMETROS	18
7.	TEMA 5: ARQUITECTURAS.....	19
7.1.	RESTRICTED BOLTZMANN MACHINES	19
7.2.	AUTOENCODERS	20
7.3.	DEEP BELIEF NETWORKS	20
7.4.	CONVOLUTIONAL NEURAL NETWORKS (CNN)	21
7.5.	RECURRENT NEURAL NETWORKS (RNN)	21
7.6.	RECURSIVE NEURAL NETWORKS	21
7.7.	GENERATIVE ADVERSARIAL NETWORKS	22
8.	TEMA 6: TENSORFLOW	22
9.	TEMA 7: INTRODUCCIÓN A RNN	26
10.	TEMA 8: RNN.....	26
11.	TEMA 9: INTRODUCCIÓN A LSTM	35
12.	TEMA 10: COMPARACIÓN DE LSTM CON RNN ESTÁNDAR Y GRU (GATED RECURRENT UNIT)	38
13.	TEMA 11: ¿CUÁNDO USAR LSTM?	40
14.	TEMA 12: LIMITACIONES Y MEJORES PRÁCTICAS	42
15.	BIBLIOGRAFÍA.....	44

16. CONCLUSIONES.....	45
-----------------------	----

1. Introducción

El Deep learning o aprendizaje profundo es una subdisciplina del Machine Learning que se centra en la creación y el entrenamiento de redes neuronales artificiales profundas. En esencia, busca imitar el funcionamiento del cerebro humano para aprender y tomar decisiones de manera autónoma.



2. Objetivos

2.1 Objetivo general

- Entender la Naturaleza del Deep Learning: Proporcionar a los estudiantes una comprensión profunda de la naturaleza del Deep Learning, explorando los principios fundamentales detrás de cómo funcionan las redes neuronales profundas, su relación con el aprendizaje automático y su uso en problemas complejos. Este entendimiento permitirá a los estudiantes apreciar el potencial y las aplicaciones del Deep Learning en campos variados.
- Conocer las Redes Neuronales: Familiarizar a los estudiantes con el funcionamiento interno de las redes neuronales, cubriendo los componentes básicos y el proceso de propagación hacia adelante y hacia atrás. Esta base permitirá a los estudiantes comprender y aplicar el proceso de entrenamiento en una red neuronal.
- Conocer las Arquitecturas Básicas del Deep Learning: Introducir a los estudiantes en las arquitecturas principales del Deep Learning, como las redes neuronales convolucionales (CNN) para imágenes, las redes neuronales recurrentes (RNN) para datos secuenciales y las redes generativas (GAN) para creación de contenido. Este conocimiento ayudará a los estudiantes a seleccionar la arquitectura adecuada para resolver diversos problemas.
- Aplicar Redes LSTM para el Procesamiento Secuencial de Datos: Enseñar el uso de las redes Long Short-Term Memory (LSTM) para abordar las limitaciones de las RNN estándar, enfocándose en cómo estas redes retienen información en secuencias largas. Los estudiantes aprenderán su utilidad en aplicaciones como el procesamiento de lenguaje natural, la predicción de series temporales y la traducción automática.
- Comprender la Estructura Interna y Funcionamiento de las Redes LSTM: Profundizar en el funcionamiento de las redes LSTM, destacando la función de la célula de memoria y los mecanismos de puertas que regulan el flujo de información. Este objetivo permitirá a los estudiantes desarrollar una comprensión completa de cómo las LSTM logran manejar dependencias a largo plazo en tareas de secuencias.

3. Tema 1: Inspiración biológica

El deep learning se inspira en la biología para crear modelos de redes neuronales artificiales que pueden aprender y realizar tareas complejas. Estas redes artificiales toman como referencia las redes neuronales del cerebro, que están formadas por aproximadamente 86 billones de neuronas y más de 500 trillones de conexiones, aunque las redes artificiales aún no alcanzan estos números. Las neuronas biológicas son unidades excitables que transmiten información a través de señales eléctricas y químicas, y aunque las neuronas artificiales imitan este comportamiento, no logran la complejidad de las neuronas reales.

Las redes neuronales artificiales tienen dos propiedades clave basadas en el cerebro:

1. Neurona artificial o nodo: Recibe información de entrada (input) que procesa y transmite a otras neuronas, a menudo aplicando transformaciones en el proceso.
2. Filtrado de información relevante: Al igual que las neuronas del cerebro, las neuronas artificiales están diseñadas para transmitir solo la información más relevante.

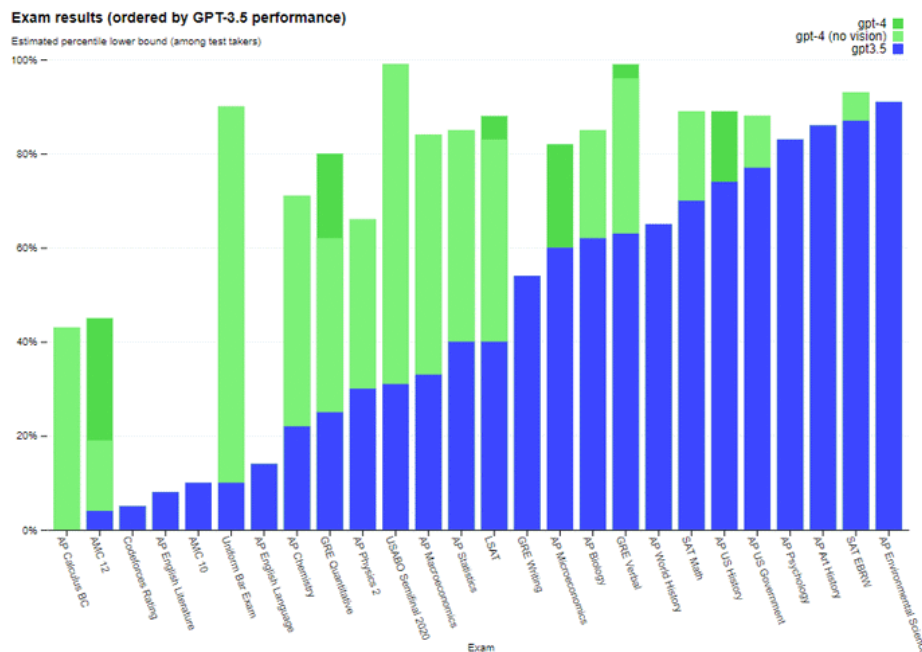
El objetivo de las redes neuronales es capturar las características esenciales del cerebro y aplicarlas en entornos artificiales, a pesar de la complejidad de este último.

4. Tema 2: Qué es el Deep Learning

El Deep Learning, basado en redes neuronales con múltiples parámetros y capas, ha ganado popularidad debido a sus impresionantes avances y resultados. Este enfoque ha demostrado superar las capacidades humanas en tareas complejas. Un ejemplo destacado es la clasificación de ImageNet, donde un estudio titulado "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification" logró un error de prueba del 4,94%, superando por primera vez el rendimiento humano en esta tarea de reconocimiento visual. Este resultado representa una mejora significativa del 26% respecto al rendimiento anterior de GoogleNet (6,66%).



1. Examen simulado de abogacía: GPT-4, desarrollado por OpenAI, ha logrado pasar un examen simulado de abogacía con una puntuación alrededor del top 10% de los participantes. En contraste, la puntuación de GPT-3.5 estaba alrededor del 10% más bajo2. ([GPT-4 \(openai.com\)](https://openai.com/gpt-4))



Estos son solo algunos ejemplos y la investigación en esta área está en constante evolución.

El mundo del Deep Learning es muy amplio y complejo, vamos a centrarnos en las principales arquitecturas existentes:

- Feedforward networks
- Redes secuenciales
- Redes convolutivas
- Redes generativas
- Redes recursivas
- Redes unsupervised

Existen muchas más arquitecturas e incluso arquitecturas híbridas pero vamos a centrarnos en las principales.

Una de las ventajas que nos proporcionan las redes neuronales es la capacidad de extraer las features clave, es decir, realizar las labores de feature extraction. Las redes son capaces de decidir qué características del dataset pueden ser utilizadas por reflejar más fielmente la información contenida en los datos. Se ha probado que su capacidad puede sobrepasar la de muchos algoritmos de Machine Learning específicos para la extracción de variables relevantes.

5. Tema 3: Matemáticas tras el Deep Learning

En esta asignatura se aborda el álgebra lineal como base fundamental del Deep Learning, centrándose en los conceptos y operaciones clave sin profundizar en los aspectos técnicos de las matemáticas. Los conceptos principales incluyen:

- Escalares: variables que almacenan un solo valor.
- Vectores: tuplas ordenadas de escalares que forman un array de valores.

- Matrices: arrays bidimensionales compuestos por vectores con la misma dimensión.
- Tensores: arrays multidimensionales.
- Hiperplanos: subespacios con una dimensión menor que el espacio original, como un plano en un espacio tridimensional.

Las operaciones matemáticas principales son:

- Producto escalar: opera con dos secuencias de igual longitud y devuelve un único número.
- Producto elemento a elemento: toma dos vectores de la misma longitud y produce un vector de la misma longitud.
- Producto exterior: multiplica cada elemento de un vector columna por todos los elementos de un vector fila.

Finalmente, se destaca la vectorización como una operación clave en Deep Learning, ya que los modelos requieren trabajar con valores ordenados en vectores, pero los datos reales no siempre están en ese formato.

Por ejemplo, podemos tener inputs de datos que sean tablas como la siguiente:

Nombre	País	Edad
Pedro	España	10
Marie	Francia	25
Linnea	Suecia	47

O textos como el siguiente:

*Deletereos de armonía
que ensaya inexperta mano.
Hastío. Cacofonía
del sempiterno piano
que yo de niño escuchaba
soñando... no sé con qué,
con algo que no llegaba,
todo lo que ya se fue.*

Deletereos de armonía, Antonio Machado

La vectorización convierte datos en vectores para que los algoritmos de Deep Learning puedan procesarlos, manteniendo la información original. Cada fila de datos se transforma en un vector donde cada valor corresponde a una columna o feature, y el conjunto completo se representa como una matriz. Por ejemplo, en un conjunto de datos con features como Nombre, País y Edad, cada uno de estos valores se convierte en un componente del vector.

En cuanto a conceptos estadísticos, se destacan tres:

- Probabilidad: mide la posibilidad de que un evento ocurra o que una proposición sea verdadera.
- Distribución: describe los posibles valores de una variable y su frecuencia de ocurrencia.
- Verosimilitud: evento con alta probabilidad de ocurrir, aunque no garantizado.

Finalmente, las técnicas de Machine Learning, como regresión, clasificación y clustering, también se aplican en el ámbito del Deep Learning.

6. Tema 4: Redes neuronales

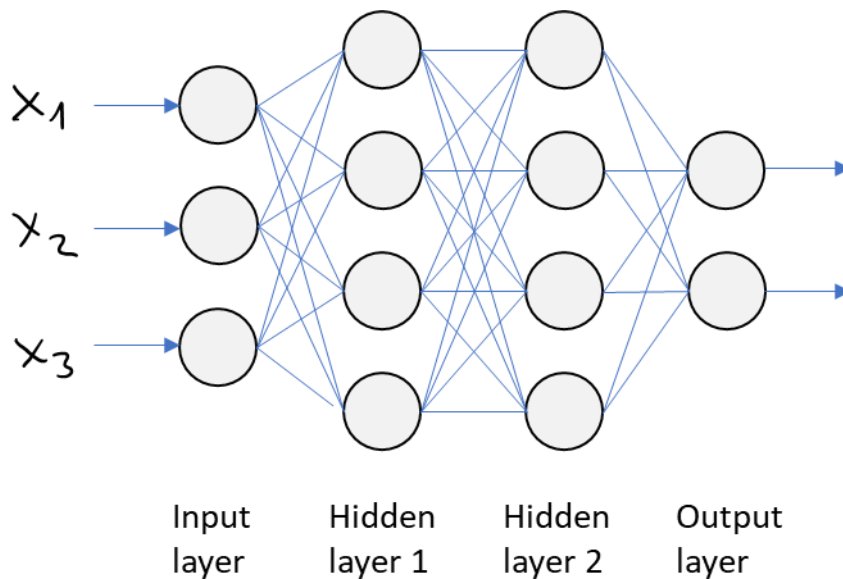
Tras un repaso de los fundamentos matemáticos que vamos a utilizar, nos adentramos en las redes neuronales. Hemos visto también como la entidad principal de una red neuronal es la neurona y éstas trabajan en paralelo y se comunican entre sí pasando información. Estas neuronas o unidades tienen entre sí unos pesos que conforman el principal medio de aprendizaje.

En términos matemáticos, podemos entenderlo como la siguiente ecuación: $Ax = b$. Donde A es la matriz de datos de entrada, b es el vector de los valores de salida y x es el conjunto de pesos.

La arquitectura de una red neuronal se define según:

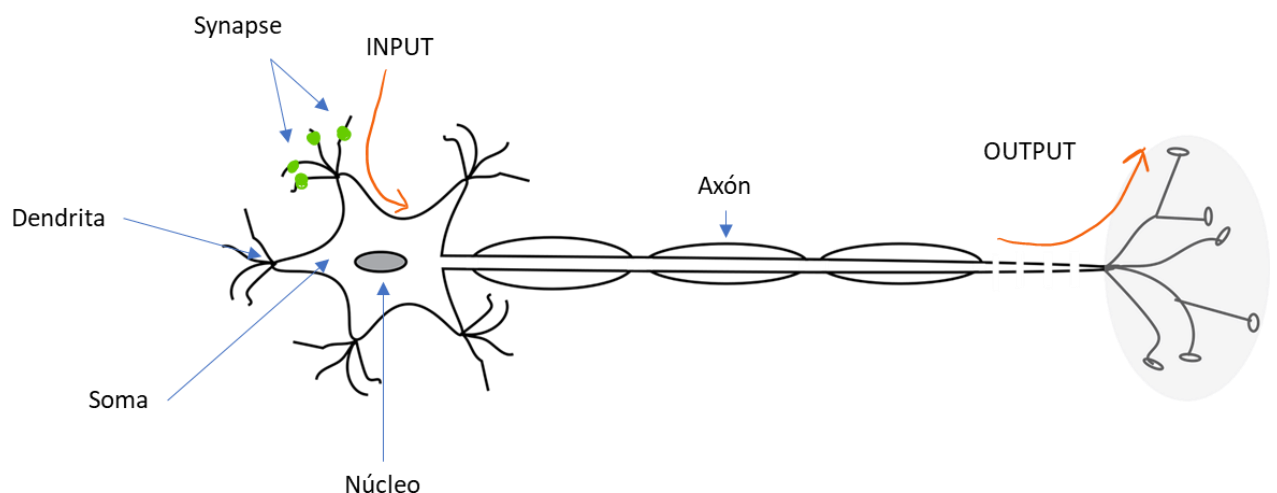
- El número de neuronas
- El número de capas (conjunto de neuronas en paralelo)
- El tipo de conexión entre las capas

La red neuronal más sencilla y básica se conoce como feed-forward multilayer neural network. Consta de una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa tiene un número variable de neuronas y cada capa está completamente conectada con la capa siguiente. Estas redes neuronales son capaces de representar cualquier función dado el número adecuado de neuronas, utiliza como algoritmo de aprendizaje el backpropagation que emplea a su vez el descenso del gradiente, lo veremos más adelante.



6.1. Neurona biológica

Antes de explicar las redes neuronales fundamentales, es necesario repasar el perceptrón, precursor de las redes neuronales actuales, inspirado en el funcionamiento de las neuronas biológicas. Una neurona es una célula que transmite impulsos electroquímicos a otras neuronas, pero solo si el impulso es lo suficientemente fuerte para superar un umbral y activar la emisión de los componentes químicos necesarios. Si el impulso no es suficiente, la información no se transmite.



Para comprender cómo los conceptos de las neuronas biológicas se aplican al deep learning, es esencial entender el funcionamiento de una neurona biológica en el cerebro humano. Una neurona está compuesta por varias partes clave: las dendritas, que reciben señales de otras

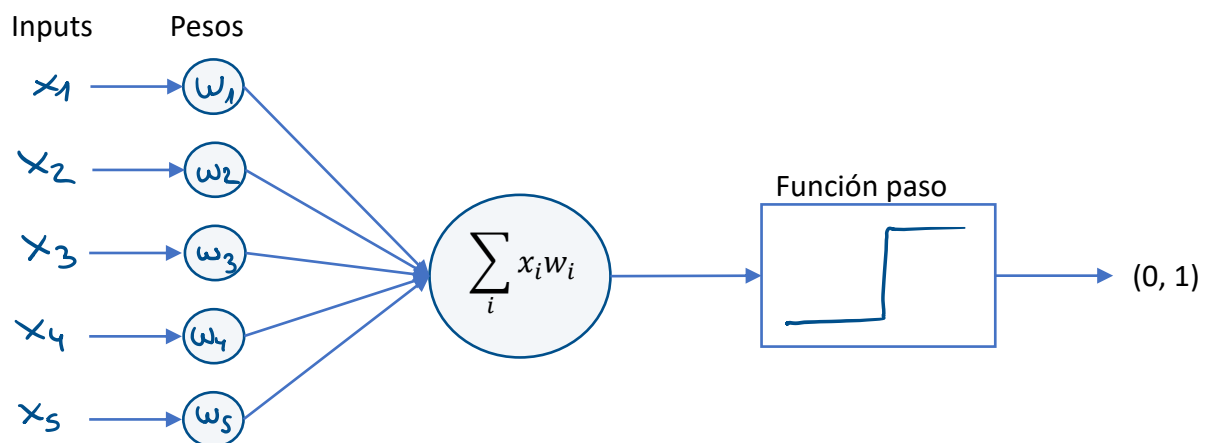
neuronas; el cuerpo celular (soma), que integra esas señales; el axón, que transmite el impulso a otras neuronas; y las sinapsis, que son las conexiones entre las neuronas donde se transmiten las señales. En resumen, las dendritas reciben señales, el soma las integra, y si superan un umbral crítico, el impulso viaja por el axón hacia otras neuronas. Este proceso se puede comparar con el funcionamiento de las neuronas artificiales en el deep learning.

6.2. Perceptrón

El perceptrón es un modelo lineal utilizado para una clasificación binaria, es una neurona artificial que utiliza el paso de Heaviside como función de activación (veremos más adelante qué son las funciones de activación).

Fue propuesto en 1957 en el Cornell Aeronautical Laboratory por Frank Rosenblatt y fue fundado por la US Office of Naval Research. Fue de mayor impacto el precursor del perceptrón, la Unidad Lógica de Umbral o Threshold Logic Unit (TLU) desarrollada por McCulloch y Pitts en 1943.

El perceptrón adquiere la siguiente forma:



El perceptrón, al igual que la neurona biológica, tiene entradas que se reciben a través de la sinapsis (pesos de entrada), que son combinadas en el cuerpo celular de la neurona (sumatorio de inputs y pesos). El axón decide si transmite la información dependiendo de la intensidad del impulso electroquímico, de forma similar a la función paso del perceptrón, que decide si el valor sumado supera un umbral para activar la salida. El algoritmo de aprendizaje ajusta los pesos hasta que todos los inputs estén correctamente clasificados, proceso que termina cuando no es posible separar linealmente las clases en el espacio de entrada. Aunque similar a la neurona biológica, el perceptrón tiene limitaciones, como la incapacidad de resolver problemas no lineales como el XOR. Este problema, que no se puede separar mediante una línea recta en el espacio de entrada, demuestra la limitación del perceptrón, aunque marca la base para el desarrollo de redes neuronales más avanzadas, como las redes feed-forward, capaces de resolver problemas no lineales.

Para comprender por qué el perceptrón no puede resolver XOR, consideremos el problema XOR. XOR es una operación lógica que toma dos entradas binarias y devuelve un resultado binario de acuerdo con la siguiente tabla:

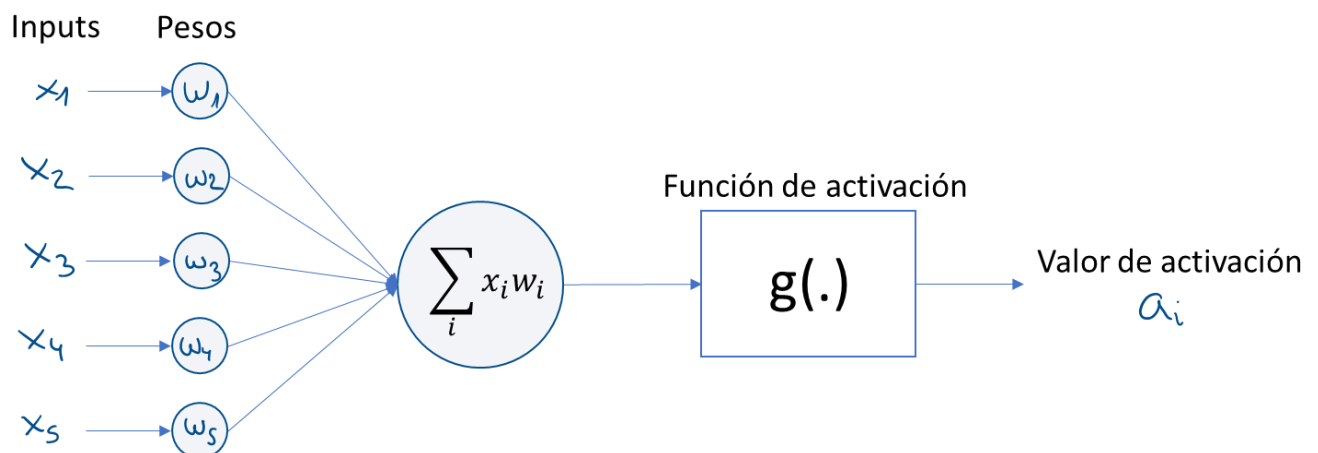
Entrada 1	Entrada 2	Salida
0	0	0
0	1	1
1	0	1
1	1	0

Como se puede observar en la tabla, los puntos de datos correspondientes a las cuatro combinaciones posibles de entradas (0,0), (0,1), (1,0) y (1,1) no pueden separarse linealmente mediante una sola línea recta en un plano bidimensional. Esto significa que no se puede encontrar un solo conjunto de pesos en un perceptrón que pueda trazar una línea recta para separar todas las combinaciones correctamente.

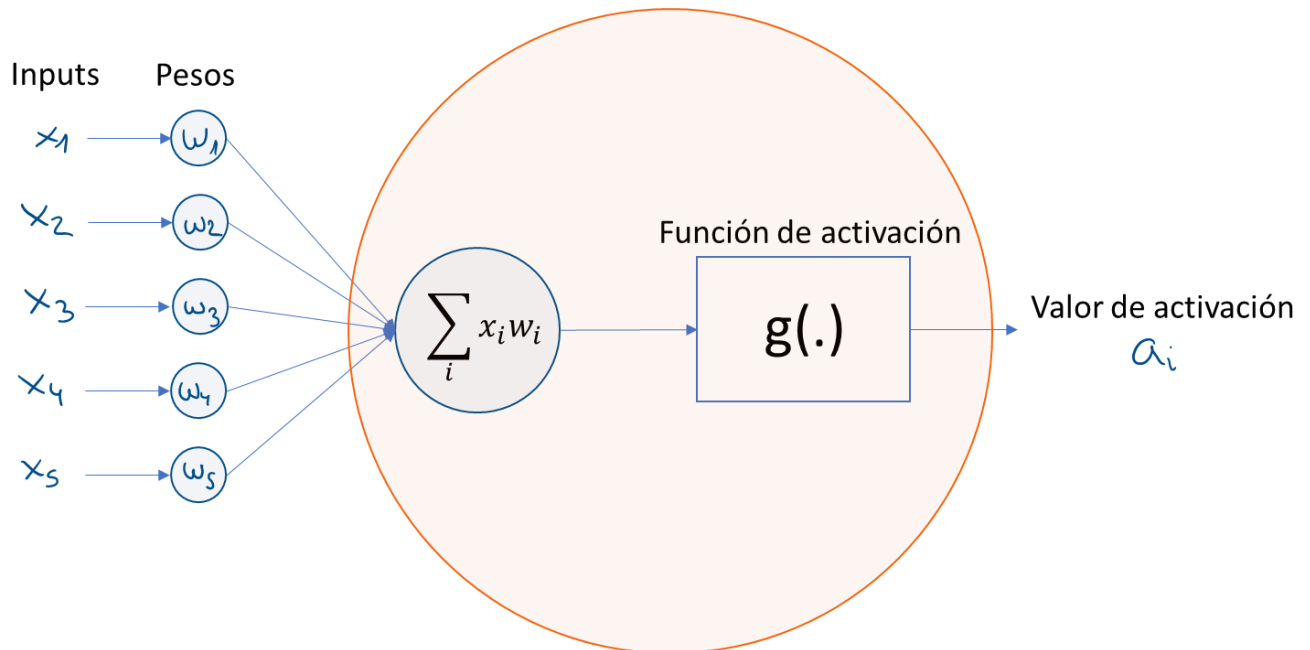
En el espacio bidimensional, las entradas correspondientes a las combinaciones (0,0) y (1,1) se encuentran en una clase (salida 0), mientras que las entradas correspondientes a las combinaciones (0,1) y (1,0) están en la otra clase (salida 1). No se puede trazar una línea recta que divida perfectamente estas dos clases en el espacio bidimensional.

6.3. Multilayer Feed-Forward Networks

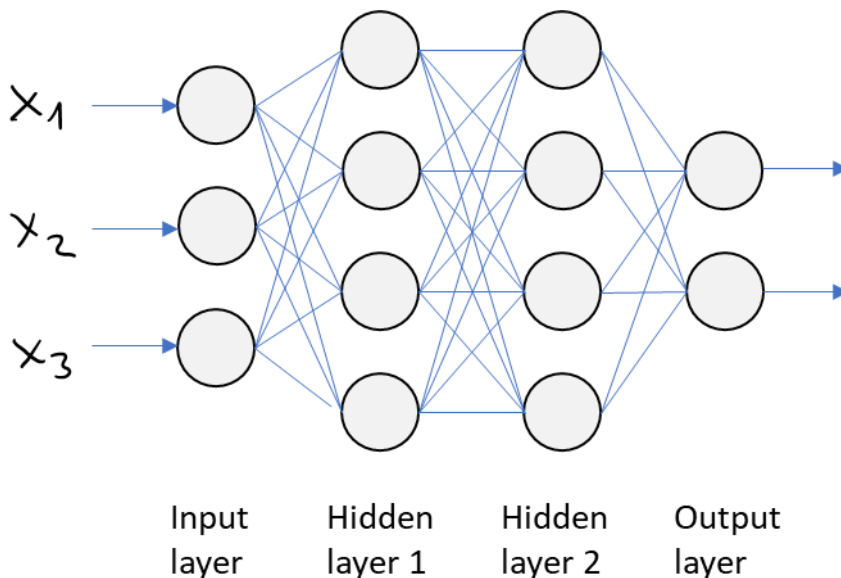
Las redes multilayer feed-forward están compuestas por una capa de entrada, una o varias capas ocultas y una capa de salida, donde cada capa tiene un número variable de neuronas y está completamente conectada con la siguiente. A diferencia del perceptrón, estas redes pueden utilizar funciones de activación diversas, que no necesariamente deben ser la función de paso, dependiendo de la función específica de cada capa dentro de la red.



Si unificamos la función de transferencia con la función de activación, obtenemos la propia neurona:



Donde $a_i = g(w_i x_i + b_i)$. Los pesos amplifican o atenúan el valor que reciben, el sesgo b garantiza que al menos un nodo por capa está activo independientemente de los inputs que le lleguen y la función de activación es la que gobierna el comportamiento de la neurona; la transmisión del input se denomina forward propagation.



En la imagen podemos ver cómo cada capa cuenta con varias neuronas.

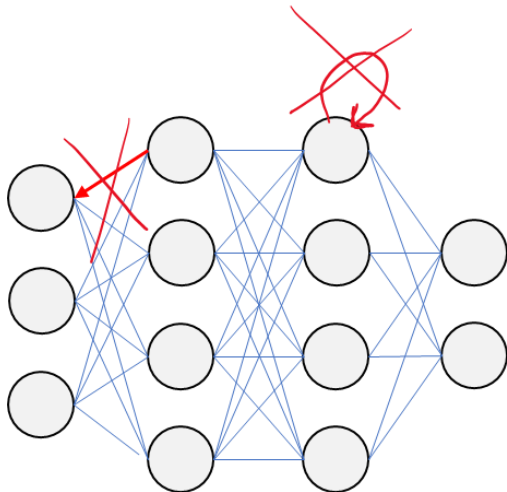
- Capa de entrada: la capa de entrada nos permite alimentar la red con los vectores entrantes, el número de neuronas en la capa de entrada suele ser el mismo número de variables de entrada. En las redes feed-forward siempre están todas las neuronas de la capa conectadas con todas las neuronas de la siguiente capa, pero esto puede no ser así en otras arquitecturas de redes neuronales.

- Capa oculta: puede haber una o más capas ocultas y son la entidad clave para poder modelar no-linearidades.

- Capa de salida: transfiere el resultado o predicción de nuestro modelo en base al valor que recibió de entrada. Dependiendo del objetivo de nuestra red, el resultado puede ser un valor real (regresión) o un conjunto de probabilidades (clasificación, siendo cada probabilidad el resultado de pertenecer a una u otra clase). El objetivo se controla en función de la función de activación que se utiliza en la capa de salida.

- Conexiones entre las capas: las conexiones definen los pesos y estos evolucionan según el algoritmo se va aproximando al resultado más preciso al aplicar el método de backpropagation.

Hemos comentado que las redes feed-forward conectan todas las neuronas de una capa con todas las neuronas de la siguiente pero no al revés, es decir, la dirección es siempre hacia adelante y tampoco hay neuronas conectadas con si mismas.



6.4. Backpropagation

El algoritmo de backpropagation entrena las redes neuronales ajustando los pesos de las neuronas según la precisión de las predicciones. Este proceso amplifica señales correctas y atenúa los ruidos. A través de la función de pérdidas, la red se ajusta premiando las predicciones correctas y penalizando las incorrectas. Inventado en 1969 por Bryson y Ho, su uso generalizado comenzó en los años 80. Durante el aprendizaje, si el resultado es incorrecto, se ajustan los pesos distribuyendo el error entre las conexiones. La tasa de aprendizaje determina la rapidez con la que se ajustan estos pesos. Aunque esencial para reducir el error, backpropagation no garantiza converger a un óptimo global ni ser completamente eficiente.

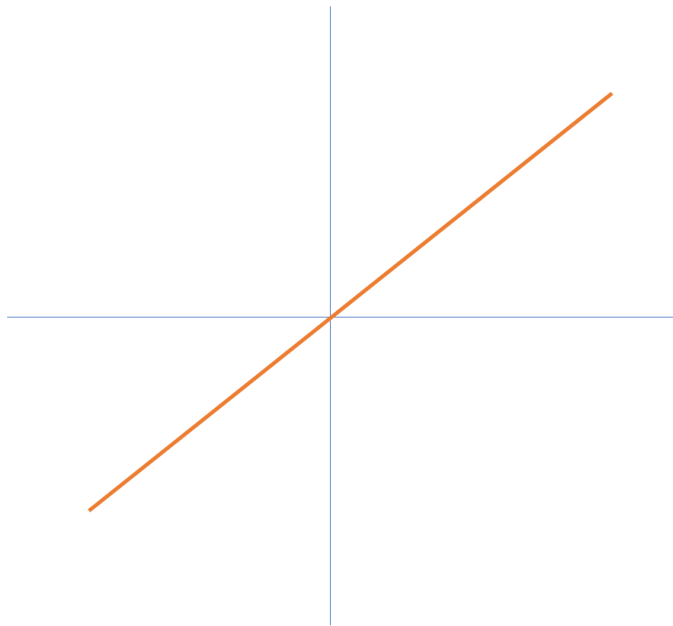
6.5. Funciones de activación

Las funciones de activación se aplican tras una capa en una red neuronal y permiten decidir si una neurona es activada o no y propagar el output de una capa a la siguiente.

Las funciones de activación nos permiten introducir no-linearidades en las capacidades de la red.

Las funciones de activación más habituales son la función lineal, sigmoid, la función tangente hiperbólica, la función softmax, la función ReLU y la función Leaky ReLU.

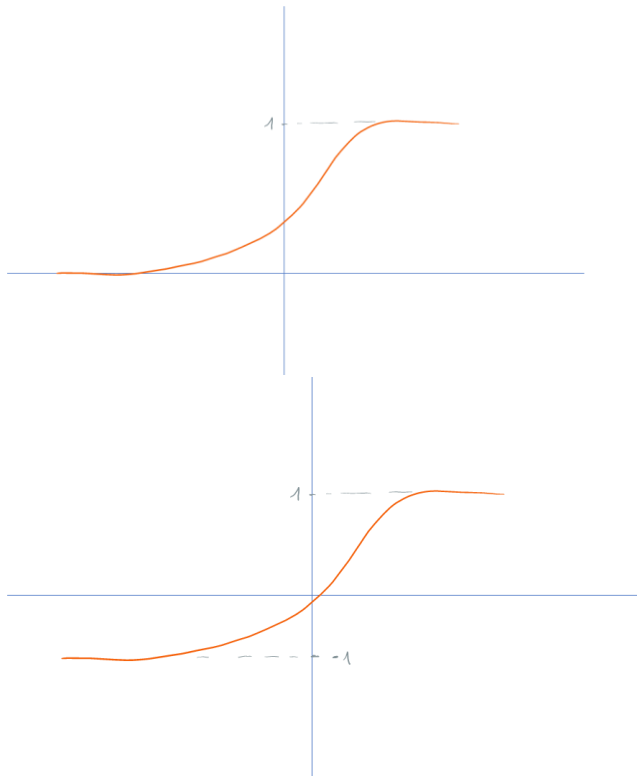
- Función lineal: Una función de activación lineal, a menudo denominada identidad, es una función matemática que simplemente devuelve su entrada sin aplicar ninguna transformación no lineal. En otras palabras, la función de activación lineal es una función lineal que no introduce ninguna no linealidad en la salida. La función de activación lineal se define de la siguiente manera: $f(x) = wx$



La función de activación lineal es simple y no modifica la forma ni la escala de la entrada, lo que resulta en una transformación lineal de los datos. Aunque no se usa frecuentemente en capas ocultas de redes neuronales profundas, se emplea comúnmente en la capa de salida para problemas de regresión, ya que permite predecir valores numéricos continuos sin restricciones.

- La función sigmoid y la tangente hiperbólica delimitan el valor de salida en un rango. Para la función sigmoid el rango es entre 0 y 1 y para la función tangente hiperbólica entre -1 y 1. Al estar acotadas, los resultados que toman valores alejados de cero, prácticamente no presentan diferencias, saturan rápido y desencadena el efecto conocido como desvanecimiento del gradiente. Por este motivo, son poco utilizadas en capas intermedias, si fueran utilizadas, el aprendizaje por medio de gradientes sería muy complicado. Sin embargo, la función sigmoid es ampliamente utilizada para problemas de clasificación binaria en la última capa.

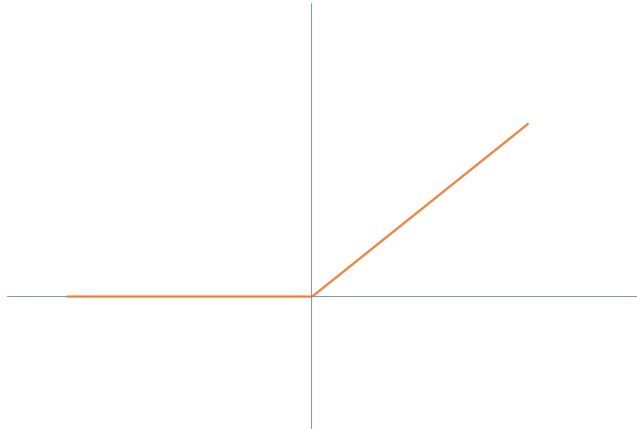
Función sigmoid: $f(x) = \frac{1}{1+e^{-x}}$



- Las funciones softmax representan una distribución de probabilidad cuando la salida es una variable discreta con más de dos valores posibles. La función sigmoid se puede entender como una particularidad de la función softmax donde el número de valores discretos posibles es dos. Su output es la distribución de probabilidad sobre las clases posibles; seleccionando el mayor de los valores, obtendremos la clase resultante como output.

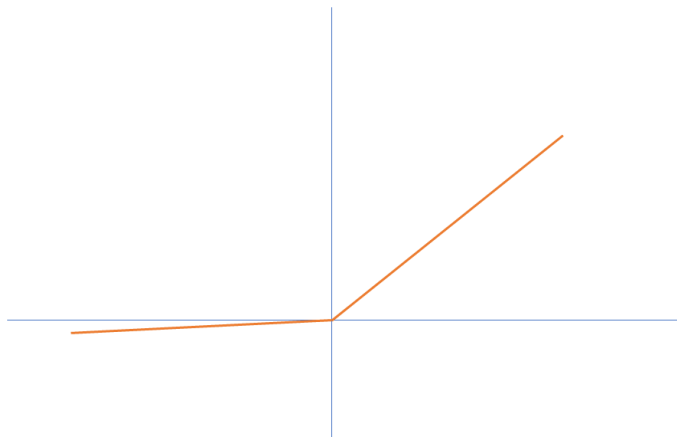
- Para resolver el problema del desvanecimiento del gradiente, una de las principales funciones alternativas a utilizar es la ReLU (Rectified Linear Unit). Estas funciones devuelven cero para valores negativos y el mismo valor para positivos, es decir, es lineal en la componente positiva. Esta característica permite que las derivadas no disminuyan y sean consistentes facilitando el proceso de aprendizaje y limitando el efecto del desvanecimiento del gradiente. Adicionalmente, al devolver como resultado cero cuando la entrada es negativa, se limita el número de neuronas activas y el proceso es más ágil y eficiente.

$$f(x) = \max(0, x)$$



- Las funciones Leaky ReLU añaden una corrección a las ReLU donde los valores negativos no valen cero sino αx donde α es un valor cercano a cero, es decir, muestran una componente lineal pero diferente a la positiva. Las funciones *Leaky ReLU* se suelen para solventar el problema conocido como *Dying ReLU* (problema desencadenado al indicar para todos los casos que el resultado es cero). Con la pequeña componente lineal α , se puede remediar dicho problema para ofrecer valores distintos a cero.

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$



6.6. Función de pérdidas

La función de pérdidas evalúa qué tan cerca está una red neuronal de su desempeño ideal, calculando el error en sus predicciones. El objetivo es encontrar los pesos que minimicen dicha pérdida, lo que convierte el entrenamiento de la red en un problema de optimización. De este modo, la función de pérdidas ayuda a estimar el éxito de la red neuronal.

Existen varios tipos de funciones de pérdidas en función del tipo de problema:

- Para las regresiones, la función de pérdidas más habitual es MSE (Minimum Squared Error), $L(w, b) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$, aunque también es habitual MAE (Mean Absolute Error),

$$L(w, b) = \frac{1}{2N} \sum_{i=1}^N \sum_{j=1}^M |\hat{y}_{ij} - y_{ij}|.$$

- Para las clasificaciones, podemos encontrar: hinge loss o negative log likelihood. Aunque tienen enfoques ligeramente diferentes y se aplican en diferentes contextos, ambos son útiles para medir el rendimiento y ajustar los modelos de clasificación. La pérdida Hinge se utiliza principalmente en problemas de clasificación binaria. Su objetivo es medir cuán bien un modelo clasifica correctamente las muestras positivas y negativas. La pérdida Hinge se define como: $L(y, f(x)) = \max(0, 1 - y * f(x))$. La Negative Log-Likelihood, también conocida como Entropía Cruzada Categórica (Categorical Cross-Entropy), se utiliza comúnmente en problemas de clasificación multiclase. Su objetivo es medir cuán bien un modelo de clasificación asigna probabilidades a las clases correctas. La pérdida de NLL se define como: $L(y, f(x)) = -\log(f(x)[y])$

- Para casos de reconstrucción, la entropía cruzada es una de las funciones de pérdidas más habituales. Los casos de reconstrucción son aquellos en los que la red neuronal no se utiliza para realizar predicciones sino para regenerar el input, es decir, la idea es tomar un input, codificarlo o reducir su dimensionalidad y regenerarlo. Algunos ejemplos son las redes generativas GAN o los autoencoders. La entropía cruzada es binaria cuando tenemos un problema de clasificación binaria y es categórica cuando tenemos multiclase.

6.7. Hiperparámetros

En Machine Learning, los hiperparámetros son parámetros que se ajustan para optimizar el entrenamiento de un modelo, controlando funciones de optimización y selección de modelo. Su objetivo es evitar el overfitting y el underfitting, buscando que el modelo aprenda lo más rápido posible sobre los datos. Los tres principales hiperparámetros que se estudian son el learning rate, las técnicas de optimización y la regularización.

- Learning rate: es un coeficiente que determina el tamaño de los pasos que da el modelo para ajustar sus parámetros. En cada iteración, el gradiente de error se multiplica por el learning rate para actualizar los pesos. A medida que el modelo converge, el learning rate generalmente se reduce.

- Técnicas de optimización:

- AdaGrad: acelera el entrenamiento en las primeras iteraciones y lo ralentiza al final. Es útil para tareas de clasificación y regresión.

- ADAM: es una técnica más reciente y eficiente, basada en el descenso de gradiente estocástico, que ajusta automáticamente la tasa de aprendizaje y acelera la convergencia.

- Regularización: se utiliza para evitar el overfitting. Existen dos tipos principales:

- L1 y L2: reducen el impacto de características poco relevantes para el modelo.

- Dropout: consiste en omitir de forma aleatoria ciertas unidades en las capas ocultas para prevenir que el modelo dependa excesivamente de ellas, acelerando el entrenamiento.

En resumen, estos hiperparámetros juegan un papel crucial en mejorar la eficiencia y precisión del entrenamiento de modelos de Machine Learning.

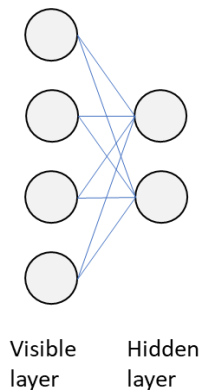
7. Tema 5: Arquitecturas

Vamos a explorar las principales arquitecturas que se pueden abordar:

- Restricted Boltzmann Machines (RBM)
 - No supervisadas:
 - Autoencoders
 - Deep Belief Networks
 - Generative Adversarial Networks
 - Convolutional Neural Networks (CNN)
 - Recurrent Neural Networks (RNN)
 - Recursive Neural Networks
-

7.1. Restricted Boltzmann Machines

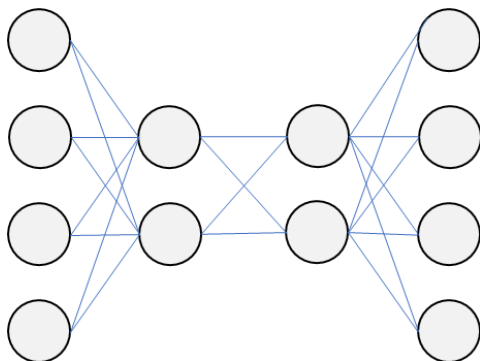
Las Restricted Boltzmann Machines (RBM) son arquitecturas utilizadas para la extracción de características y reducción de dimensionalidad, donde la palabra "restricted" indica que no hay conexiones entre nodos de la misma capa. Fueron popularizadas por Geoff Hinton y se consideran una forma de autoencoders. Se componen de tres elementos principales: unidades visibles, unidades ocultas y pesos, siendo las unidades ocultas los detectores de características de los datos de entrada. En el preentrenamiento, las RBM aprenden a reconstruir los datos de entrada a partir de una muestra limitada, trabajando en un entorno no supervisado (unsupervised) para realizar ingeniería de características. Los pesos obtenidos se utilizan para inicializar redes como las Deep Belief Networks, y el cálculo de gradientes se realiza mediante el algoritmo de contrastive divergence.



7.2. Autoencoders

Los autoencoders son modelos utilizados para aprender representaciones comprimidas de conjuntos de datos, especialmente en tareas de reducción de dimensionalidad. Su objetivo es reconstruir los datos de entrada de manera más eficiente. Estos modelos, similares a las redes feed-forward, tienen capas de entrada, salida y ocultas, pero con el mismo número de nodos en la capa de entrada y salida. A diferencia de las redes tradicionales, los autoencoders no requieren etiquetas (aprendizaje no supervisado) y pueden generar versiones comprimidas de los datos. Utilizan backpropagation para actualizar los pesos, aunque con un cálculo diferente de los gradientes.

Existen dos variantes principales de autoencoders: los compression autoencoders, que comprimen los datos a través de un "cuello de botella", y los denoising autoencoders, que aprenden a eliminar el ruido de datos corruptos. Aunque al principio la idea de construir un modelo para representar los datos de entrada puede no parecer útil, cobra relevancia cuando se analiza la diferencia entre la entrada y la salida. Esto permite a la red identificar datos inusuales o anómalos, lo que convierte a los autoencoders en una herramienta eficaz para la detección de anomalías. Además, suelen ser integrados en redes más complejas en lugar de usarse de forma aislada.



7.3. Deep Belief Networks

Las Redes Bayesianas Profundas (DBN) combinan capas de Restricted Boltzmann Machines (RBM) para la fase de preentrenamiento y una red feed-forward para el fine-tuning. En el preentrenamiento, las RBMs extraen características complejas de los datos al ajustar los pesos para reconstruir las entradas. Durante el fine-tuning, se emplea una red feed-forward con retropropagación y un aprendizaje de tasa baja, con el objetivo de especializar la red en tareas específicas, como la regresión. Las capas de la red aprenden a reconstruir distribuciones de probabilidad de las activaciones previas. Aunque efectivas, las DBN han sido superadas por las Redes Neuronales Convolucionales (CNN).

7.4. Convolutional Neural Networks (CNN)

Las redes convolutivas son un tipo de redes neuronales inspiradas en la corteza visual de los animales, utilizadas para procesar datos con estructura espacial, como imágenes o señales de audio. Están compuestas por capas que realizan operaciones de convolución, activación, agrupación y conexión completa para extraer características locales y generar representaciones abstractas. Se entrenan mediante algoritmos de retropropagación, optimización y pérdida, y emplean técnicas de regularización para evitar el sobreajuste. Estas redes se aplican en áreas como visión artificial, procesamiento del lenguaje natural, reconocimiento de voz, detección de anomalías y descubrimiento de fármacos. Se mencionan brevemente, ya que se abordarán más a fondo en futuras sesiones.

7.5. Recurrent Neural Networks (RNN)

Las redes neuronales recurrentes (RNN) son un tipo de red dentro de la familia de redes feed-forward, pero se distinguen por su capacidad para procesar información a lo largo del tiempo, permitiendo la computación tanto en paralelo como secuencial. Su característica principal es su habilidad para modelar la dimensión temporal. En este caso, no se profundizará más en este tema, ya que se dedicarán sesiones exclusivas a este tipo de redes.

7.6. Recursive Neural Networks

Las redes neuronales recursivas son un tipo de red capaz de trabajar con entradas de longitud variable, diferenciándose de las redes neuronales recurrentes al modelar estructuras jerárquicas en los datos. Estas redes son útiles para descomponer escenas complejas, como las imágenes con múltiples objetos, ya que permiten detectar objetos y entender sus relaciones. Su arquitectura incluye una matriz de pesos compartidos y una estructura de árbol binario, lo que facilita el aprendizaje de secuencias variables en imágenes o palabras. Utilizan una variante del algoritmo de retropropagación llamada backpropagation through structure (BPTS). Un tipo de red recursiva es el recursive autoencoder, que aprende a reconstruir la entrada. Las redes recursivas comparten aplicaciones con las redes recurrentes, como el procesamiento del lenguaje natural, y son útiles para descomponer escenas de imágenes o frases en componentes más pequeños. Ejemplos de aplicaciones incluyen los autoencoders recursivos para segmentar frases y los recursive neural tensors para identificar y etiquetar objetos en imágenes.

7.7. Generative Adversarial Networks

Las redes adversariales generativas (GAN) se emplean principalmente para la creación de contenido, especialmente imágenes, aunque también se aplican en audio y vídeo. Estas redes utilizan un enfoque de aprendizaje no supervisado, donde se entrenan dos modelos en paralelo: el generador y el discriminador. El generador crea imágenes a partir de un conjunto de datos, como ImageNet, mientras que el discriminador clasifica las imágenes generadas para determinar si son reales o falsas. El modelo se considera eficaz cuando el generador engaña al discriminador, haciéndole creer que las imágenes generadas son reales.

El generador ajusta sus parámetros mediante retroalimentación del discriminador en cada iteración del entrenamiento. Este proceso busca aprender a representar eficientemente las características de los datos. Las GAN son modelos complejos que manejan grandes volúmenes de datos y parámetros, como los 200GB de imágenes y 100MB de parámetros de ImageNet.

El discriminador suele ser una red convolucional, mientras que el generador utiliza capas deconvolucionales, que son claves para crear imágenes realistas. Estas capas mapean características a píxeles y se entrenan en secuencia. Además, existen variantes como las GANs condicionales, que emplean información etiquetada para generar imágenes específicas de una categoría.

8. Tema 6: Tensorflow

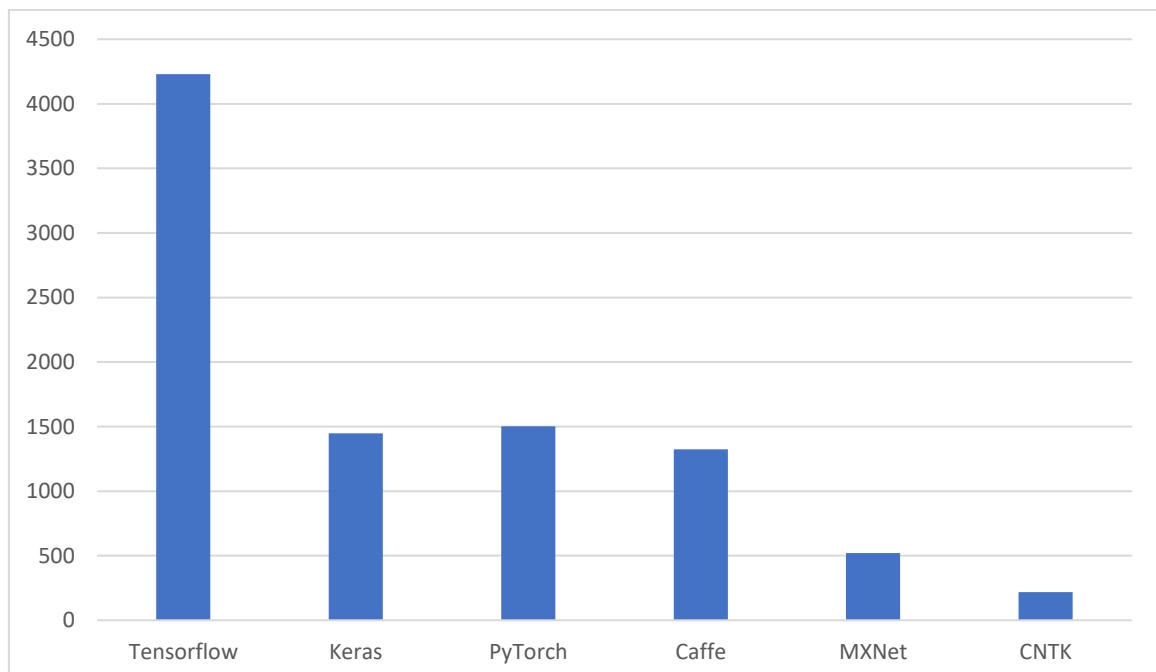
TensorFlow es una biblioteca de código abierto para cálculos numéricos basada en grafos de flujo de datos, donde los nodos representan operaciones matemáticas y los bordes son tensores (arrays multidimensionales). Desarrollada por Google Brain, está diseñada para investigaciones sobre aprendizaje automático y redes neuronales. Ofrece una arquitectura flexible que permite ejecutar cálculos en CPUs, GPUs o clusters, y es compatible con una API multilenguaje. Su implementación en C++ le da alta velocidad, y proporciona soporte integrado para Deep Learning, facilitando la creación y entrenamiento de redes neuronales.

Comparativa de frameworks de Deep Learning

Software	Software license ^[6]	Open source	Platform	Written in	Interface	OpenMP support	OpenCL support	CUDA support	Automatic differentiation ^[1]	Has pretrained models	Recurrent nets	Convolutional nets	RBM/DBNs	Parallel execution (multi node)
Keras	MIT license	Yes	Linux, macOS, Windows	Python	Python, R	Only if using Theano as backend	Can use Theano, Tensorflow or PlaidML as backends	Yes	Yes	Yes ^[17]	Yes	Yes	No ^[18]	Yes ^[19]
MATLAB + Deep Learning Toolbox	Proprietary	No	Linux, macOS, Windows	C, C++, Java, MATLAB	MATLAB	No	No	Train with Parallel Computing Toolbox and generate CUDA code with GPU Coder ^[20]	Yes ^[21]	Yes ^{[22][23]}	Yes ^[22]	Yes ^[22]	Yes	With Parallel Computing Toolbox ^[24]
Apache MXNet	Apache 2.0	Yes	Linux, macOS, Windows, ^{[25][27]} AWS, Android, ^[38] iOS, JavaScript ^[39]	Small C++ core library	C++, Python, Julia, Matlab, JavaScript, Go, R, Scala, Perl, Clojure	Yes	On roadmap ^[40]	Yes	Yes ^[41]	Yes ^[42]	Yes	Yes	Yes	Yes ^[43]
PlaidML	AGPL	Yes	Linux, macOS, Windows	Python, C++, OpenCL	Python, C++	?	Some OpenCL ICDs are not recognized	No	Yes	Yes	Yes	Yes		Yes
PyTorch	BSD	Yes	Linux, macOS, Windows	Python, C, C++, CUDA	Python, C++	Yes	Via separately maintained package ^{[44][45][46]}	Yes	Yes	Yes	Yes	Yes		Yes
TensorFlow	Apache 2.0	Yes	Linux, macOS, Windows, ^[48] Android	C++, Python, CUDA	Python (Keras), C/C++, Java, Go, JavaScript, R, ^[47] Julia, Swift	No	On roadmap ^[48] but already with SYCL ^[49] support	Yes	Yes ^[50]	Yes ^[51]	Yes	Yes	Yes	Yes
Wolfram Mathematica	Proprietary	No	Windows, macOS, Linux, Cloud computing	C++, Wolfram Language, CUDA	Wolfram Language	Yes	No	Yes	Yes	Yes ^[67]	Yes	Yes	Yes	Yes ^[68]

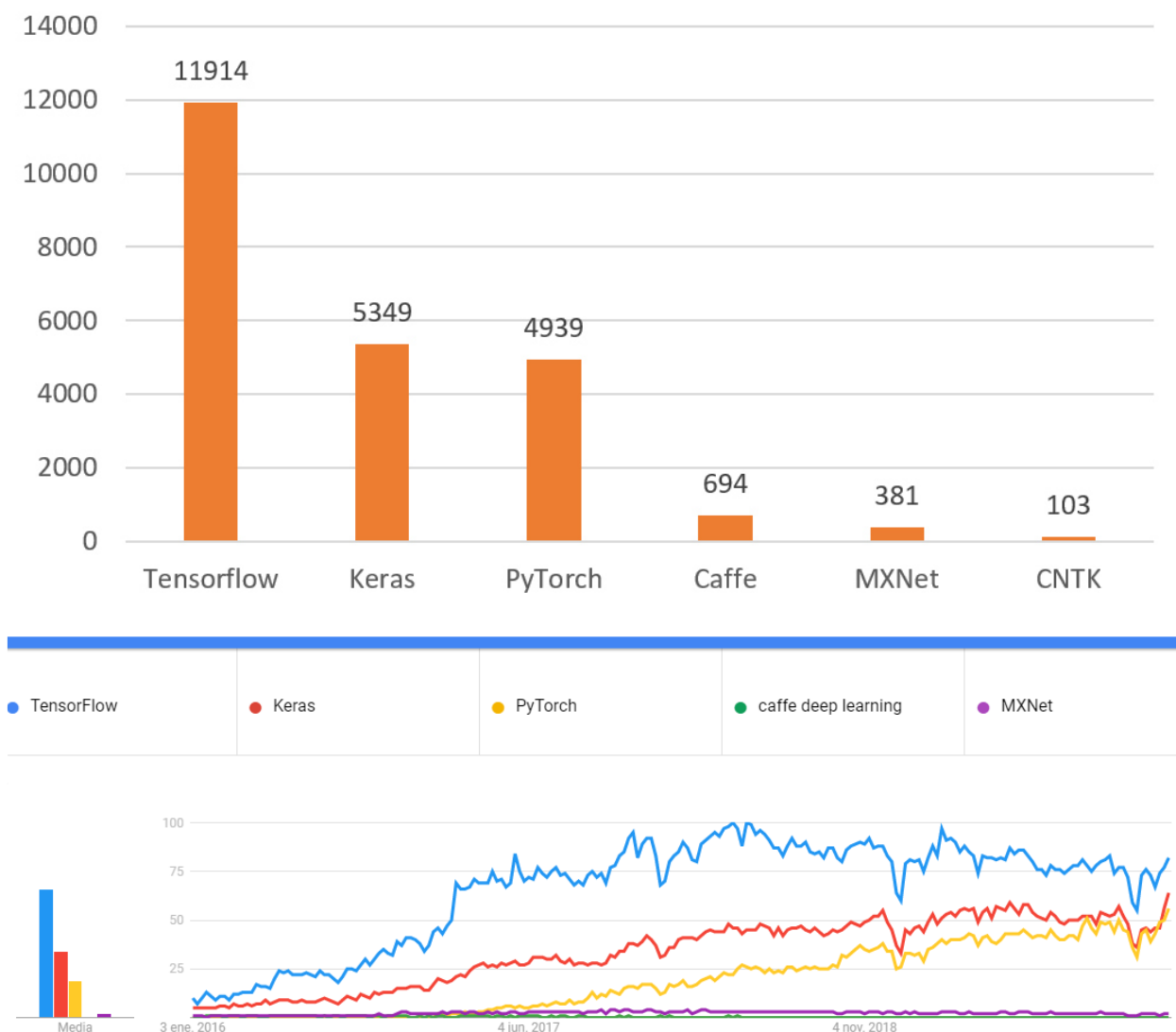
Uso de los principales frameworks de Deep Learning

Esta gráfica muestra el número de trabajos publicados en LinkedIn según cada uno de los frameworks de Deep Learning.



Proyectos públicos de GitHub que utilizan Tensorflow

No. of Public Github Projects



Instalación y configuración

El proceso de instalación de TensorFlow puede variar dependiendo de varios factores. Esto incluye si quieres soporte de GPU, o sólo de CPU, o si quieres usar Python estándar o distribuciones como Anaconda. Veremos cómo instalarlo en un entorno virtual para Python3 con Anaconda y sólo para CPU. De momento instalaremos la versión de sólo CPU porque la instalación de la versión de GPU requiere tener una GPU compatible con una correcta instalación de su controlador, Cuda Toolkit y CUDNN.

Primero descargaremos e instalaremos la última versión de Ubuntu LTS (20.04) en nuestra máquina (podemos usar un software de virtualización para crear una máquina virtual si es necesario). Luego descargamos la última versión de Anaconda para Python 3 para Linux:

<https://www.anaconda.com/distribution/#download-section>

Y lo instalamos con este comando donde X, Y, Z representa la versión que acabamos de descargar.

```
$ bash Anaconda3-XYZ-Linux-x86_64.sh
```

Una vez que tengas todo instalado, creamos un entorno virtual (que llamaremos "tensorflow") y lo activamos:

```
$ conda create -n tensorflow pip python=3.7
```

```
$conda activate tensorflow
```

Y con todo el entorno configurado podemos ahora instalar TensorFlow:

```
(tensorflow) $ pip install tensorflow
```

Cuando terminemos de usar TensorFlow podemos desactivar el entorno si queremos:

```
(tensorflow)$ conda deactivate
```

Y si queremos usarlo de nuevo, sólo tenemos que activarla de nuevo:

```
(tensorflow)$ conda activate tensorflow
```

Como consejo, si se va a trabajar con notebooks de Jupyter, se recomienda instalar ipython dentro del entorno, ya que, si no utiliza el de base, donde tensorflow no estará disponible.

Para comprobar si TensorFlow fue instalado correctamente, podemos iniciar una Shell de Python e intentar importarlo. ¡Asegúrate de activar primero el entorno que acabas de crear! Si el siguiente código no da ningún mensaje de error, entonces podemos concluir que TensorFlow fue instalado correctamente:

```
$python
```

```
>>>import tensorflow as tf
```

Notas sobre la instalación

Las GPUs son mucho más eficientes que las CPUs en el entrenamiento e inferencia de redes neuronales, ya que algunos algoritmos serían demasiado lentos si se ejecutaran solo en CPU. Dado que no todos disponen de una GPU potente, inicialmente se usará la versión de TensorFlow para CPU. Anaconda es una distribución de Python que incluye herramientas útiles y el administrador de paquetes "conda". Aunque Ubuntu LTS es la referencia para el aprendizaje profundo, es posible instalar Anaconda y TensorFlow en Windows y otras distribuciones de Linux, aunque no se recomienda.

9. Tema 7: Introducción a RNN

Las Redes Neuronales Recurrentes (RNNs) son un tipo de red neuronal especializada en procesar datos secuenciales, lo cual es crucial para tareas donde el orden de los datos importa, como el procesamiento de lenguaje natural, la predicción de series temporales y la transcripción de audio a texto. A diferencia de las redes neuronales tradicionales, las RNNs pueden "recordar" información previa, permitiendo que cada salida dependa no solo de la entrada actual, sino también de pasos anteriores. Esto las hace útiles para tareas que requieren contexto, como la traducción automática y el análisis de sentimiento.

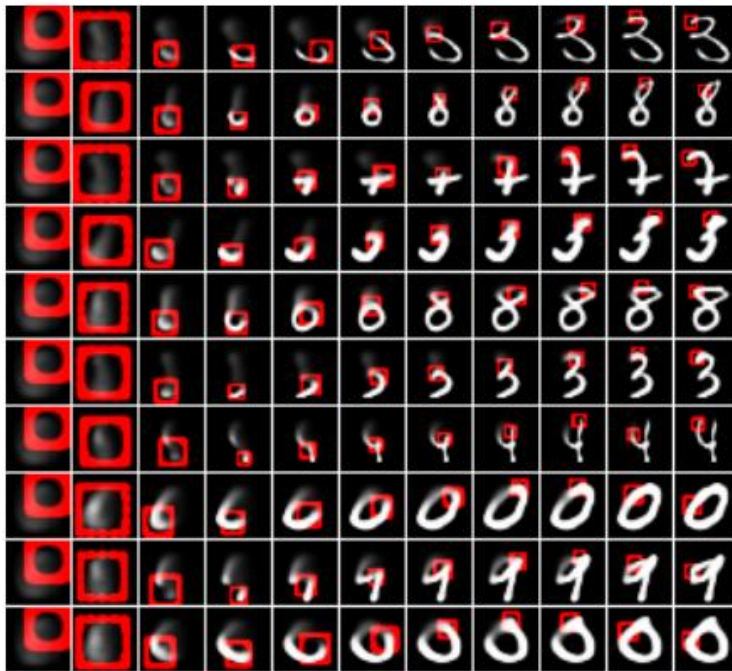
Sin embargo, las RNNs estándar presentan limitaciones, principalmente debido al problema de desvanecimiento del gradiente. Este fenómeno dificulta el aprendizaje de dependencias a largo plazo, ya que los gradientes se vuelven pequeños al retroceder en el tiempo, haciendo que la red olvide eventos lejanos. Además, existe el problema opuesto de la explosión del gradiente, que también afecta la estabilidad del entrenamiento.

Para resolver estas dificultades, se desarrollaron arquitecturas como LSTM (Long Short-Term Memory) y GRU (Gated Recurrent Units), que gestionan mejor la información a lo largo del tiempo. Aunque las RNNs estándar son limitadas en secuencias largas, siguen siendo fundamentales en el aprendizaje profundo y sientan las bases para redes más avanzadas.

10. Tema 8: RNN

Hasta ahora, los modelos que hemos estudiado se han basado en redes feed forward, que siguen una estructura simple de entrada, procesamiento en una "caja negra" y salida. En una tarea de clasificación binaria, el resultado sería la clase seleccionada, y en una no binaria, se obtendría un conjunto de probabilidades de pertenencia a cada clase. Sin embargo, en muchos casos, necesitamos que el modelo pueda manejar tipos de datos de entrada y salida más diversos, y es aquí donde las redes recurrentes (RNN) resultan útiles.

A diferencia de las redes feed forward, las RNN permiten configuraciones de entrada y salida más flexibles, como uno a N, N a uno o N a N. Por ejemplo, una configuración uno a N podría ser generar un texto de varias palabras a partir de una imagen; N a uno podría ser analizar el sentimiento de un texto de varias palabras, y N a N podría ser una traducción o interacción en un chat. Estas configuraciones permiten manejar secuencias y adaptar la entrada y salida según el contexto, lo cual es de gran utilidad para diversas aplicaciones.

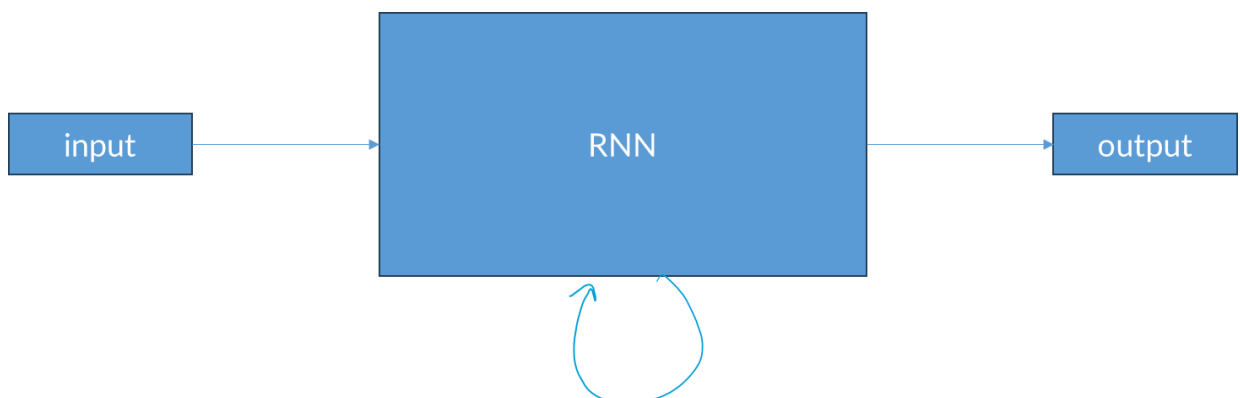


En la imagen, podemos ver un paper donde se aplican las redes recurrentes para el caso de uso de generar imágenes, en este caso números, y vemos cómo según avanzan las iteraciones hacia la derecha, se va generando el número.

[DRAW: A Recurrent Neural Network For Image Generation](#)

La arquitectura de las redes recurrentes (RNN) es bastante sencilla. Toman un dato de entrada y lo procesan a través de un estado interno oculto que se actualiza cada vez que la red recibe una nueva entrada. Este estado oculto se transfiere al siguiente paso, permitiendo que la red mantenga una "memoria" a lo largo de la secuencia. En muchos casos, se desea que la red también produzca una salida en cada paso. Así, el patrón general de funcionamiento es: recibir un input, actualizar el estado oculto y generar un output en cada paso de tiempo.

Ahora, ¿Cómo podemos representar esta misma idea mediante una función?

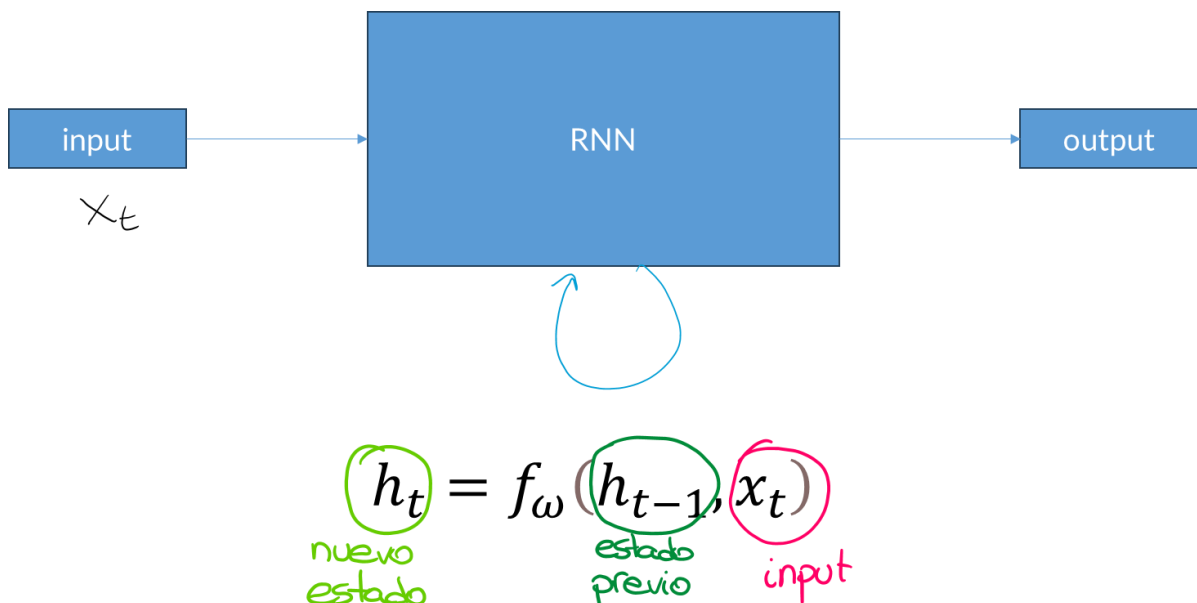


Dentro de este gran bloque de RNN, estamos calculando una relación de recurrencia con una función f , esta función f dependerá de algunos pesos W y tomará el estado anterior (h_{t-1}) y el input de entrada en el instante actual (x_t) y esto generará el siguiente nuevo estado oculto actualizado, h_t .

Cuando nos llegue el siguiente input x_{t+1} , este lo utilizaremos junto con el estado h_t que acabamos de calcular.

Si ahora queremos generar también un output, podemos conectar algunas capas fully connected que lean este estado h_t en cada instante temporal y tomaremos la decisión del output en función del hidden state de cada instante.

Algo importante a tener en cuenta, es que, en cada instante temporal utilizamos la misma función f y los mismos pesos W en cada paso.



Si buscamos entender esta función f qué tiene realmente por dentro, podemos entenderla partiendo de la red recurrente más básica.

La forma más sencilla de entenderlo sería que tenemos una matriz W_{xh} que multiplicamos por el input x_t , y, por otro lado, tenemos una matriz W_{hh} que multiplicamos por el hidden state previo. Sumamos ambos resultados y los aplicamos sobre una función de tangente hiperbólica para obtener una no linealidad sobre el sistema.

$$h_t = f_\omega(h_{t-1}, x_t)$$

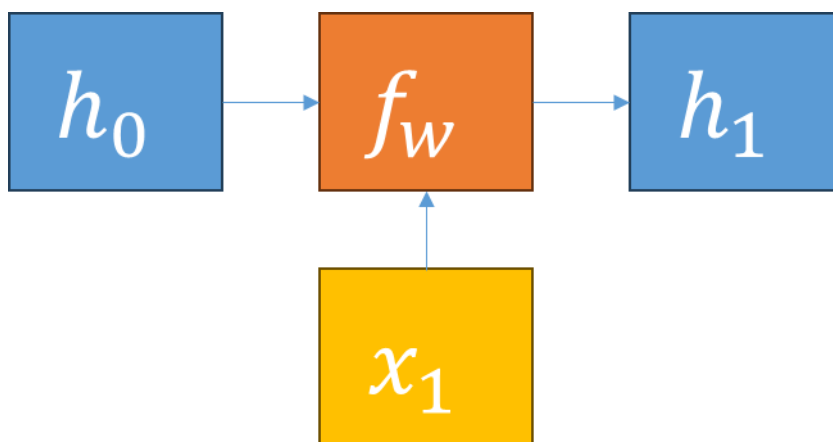
UAX.COM $h_t = \tanh(w_{hh}h_{t-1} + w_{xh}x_t)$

© Universidad Alfonso X el Sabio

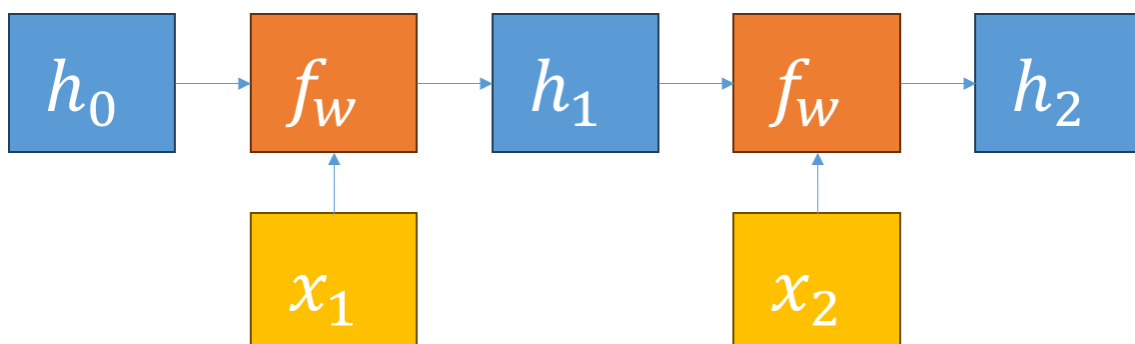
Podríamos tener otra matriz de pesos W que acepte este vector de estado oculto y lo transforme generando el output y .

$$y_t = w_{hy}h_t$$

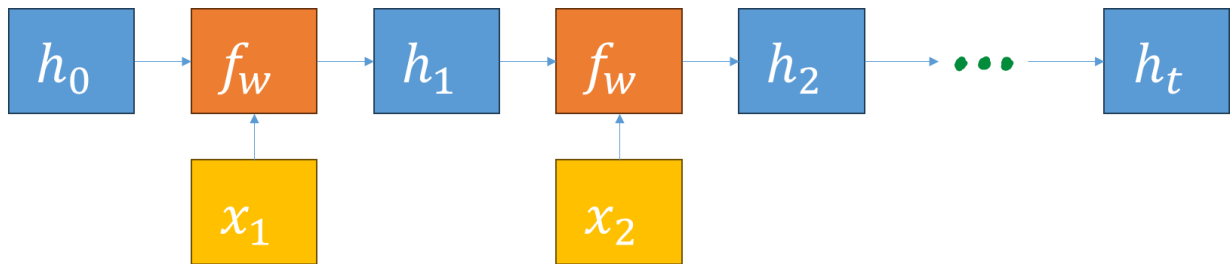
Las redes recurrentes se pueden entender de dos maneras: como un estado oculto que se retroalimenta recurrentemente o como una secuencia desplegada en varios instantes temporales, lo cual facilita la visualización del flujo de datos entre estados ocultos, entradas, salidas y pesos. En el instante inicial, se establece un estado oculto h_0 , generalmente inicializado en ceros. Luego, con una entrada x_1 , este estado y la entrada se pasan a una función f_w , que genera el siguiente estado oculto h_1 .



Luego, repetiremos este proceso cuando recibamos la siguiente entrada x_2 . Ahora, nuestro h_1 actual y nuestro x_2 actual entrarán en la función f_w para generar h_2 .



Este proceso se repetirá una y otra vez a medida que consumamos todas las entradas x .

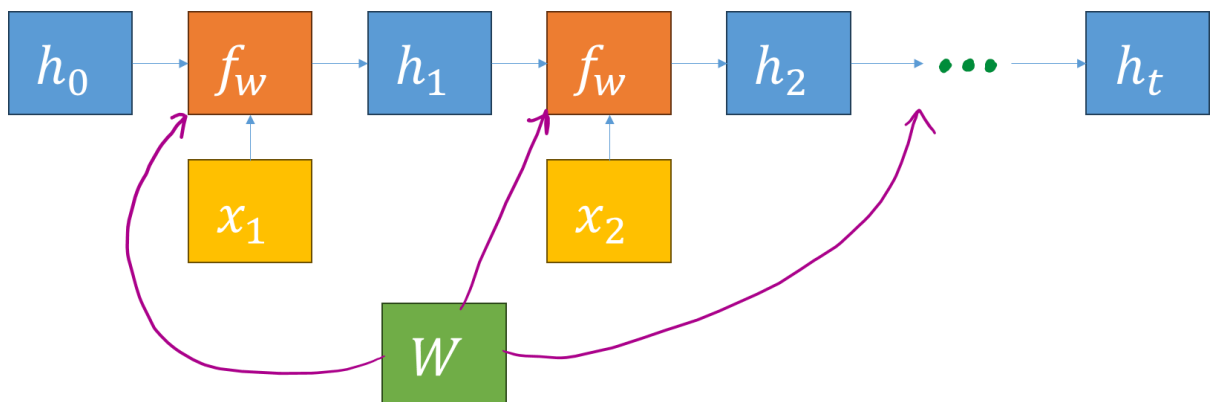


Algo que podemos añadir a esta explicación para que sea más explícita sería añadir la matriz w que utilizamos.

Aquí podemos ver que se está reutilizando la misma matriz W para todas las iteraciones por lo que cada vez que tenemos este bloque f_w , este recibe un estado oculto h y un input x nuevo cada vez pero la matriz de pesos W es siempre la misma.

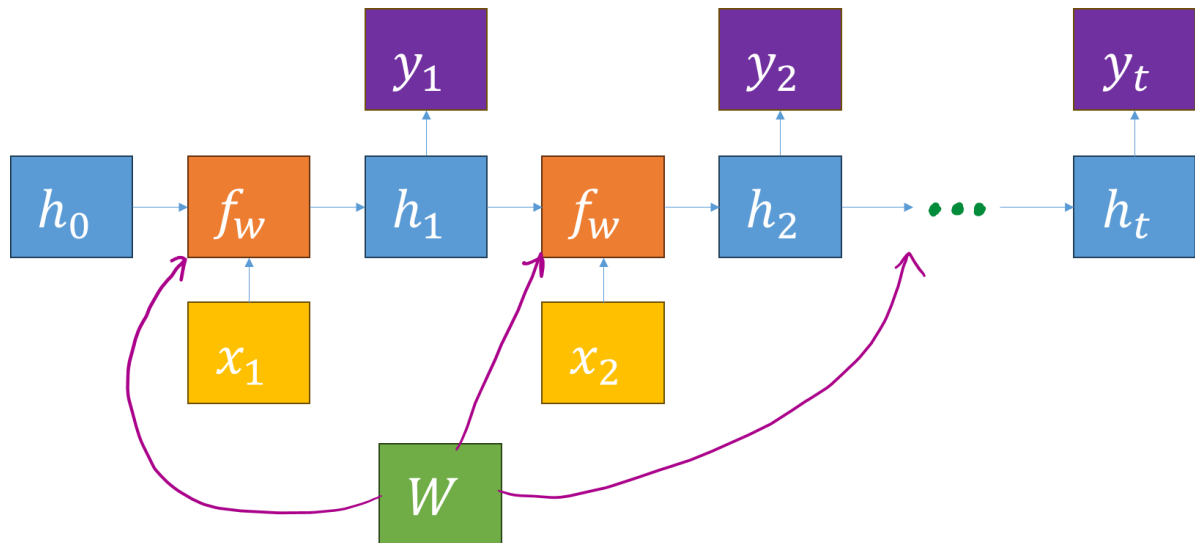
Si recordamos la operación de backpropagation, cuando se aplica, la información fluye hacia atrás para transmitir qué puntos corregir y, para ello, una operación que se utiliza es el gradiente.

En esta representación, cada instante temporal genera un valor de gradiente diferente, por lo que para entender la corrección a aplicar sobre W , el gradiente para w será la suma de todos esos gradientes individuales en cada instante temporal.



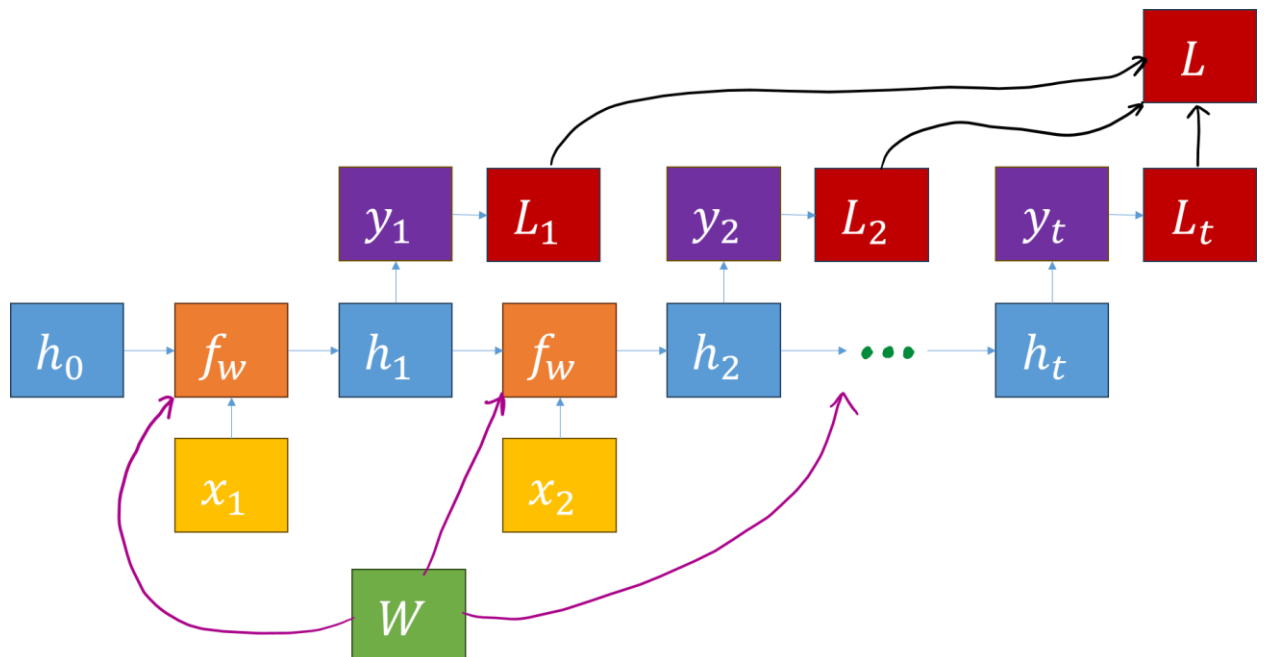
Una de las piezas que nos falta es el dato de salida y .

La salida h en cada paso de tiempo podría alimentar alguna otra pequeña red neuronal que puede producir y .



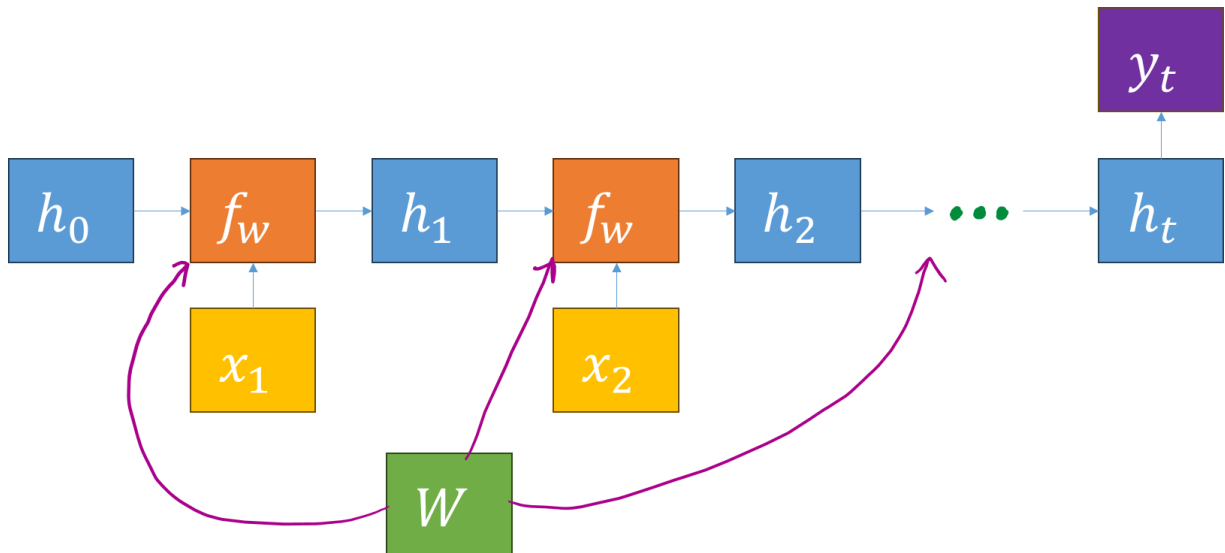
A su vez, en cada una de estas salidas y , podemos calcular una pérdida que, si las sumamos, generarán la pérdida total.

Si volvemos al backpropagation sobre esta pérdida, para entrenar el modelo, tenemos que calcular el gradiente de la pérdida con respecto a w . Tendremos una pérdida que fluirá desde esa pérdida final hacia cada uno de los instantes temporales y luego, cada uno de los instantes temporales, calculará un gradiente local en los pesos w que luego se sumará para darnos nuestro gradiente final para los pesos w .

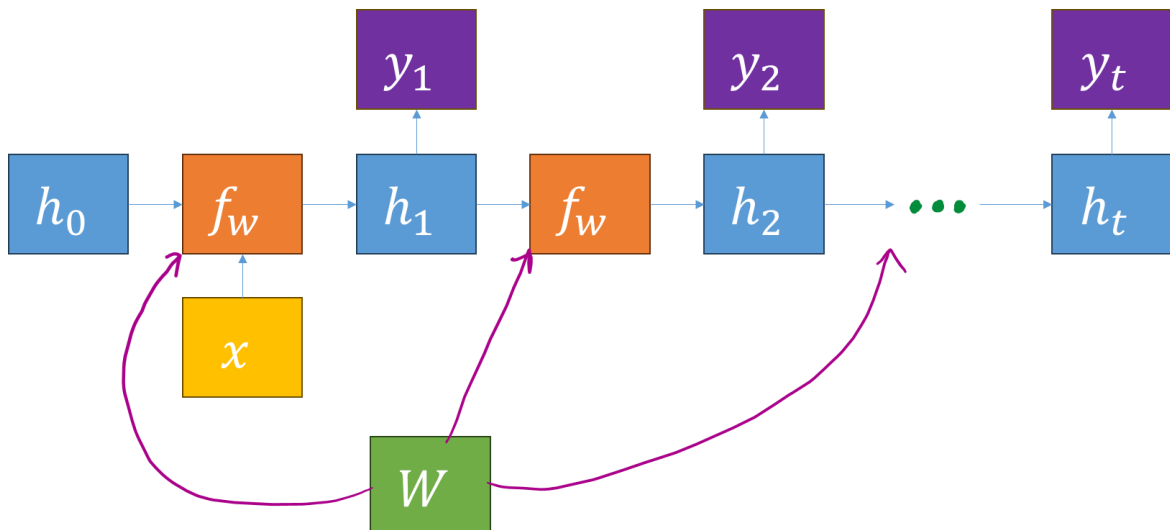


Toda esta arquitectura, aplica para un caso de N a N, es decir, nuestro vector x tenía más de un valor y nuestro vector y también.

Si ahora tenemos un escenario de N a 1, en la que tal vez queramos hacer algo como un análisis de sentimiento, entonces normalmente tomaríamos esa decisión basándonos en el estado oculto final ya que este resume todo el contexto de toda la secuencia.



Si tenemos una situación de una a N, en la que queremos recibir una entrada de tamaño fijo y luego producir una salida de tamaño variable. Normalmente se usará esa entrada de tamaño fijo para inicializar el estado oculto inicial del modelo y ahora la red recurrente marcará cada celda en la salida y ahora, a medida que produce su resultado de tamaño variable desenrollará el gráfico para cada elemento del resultado.

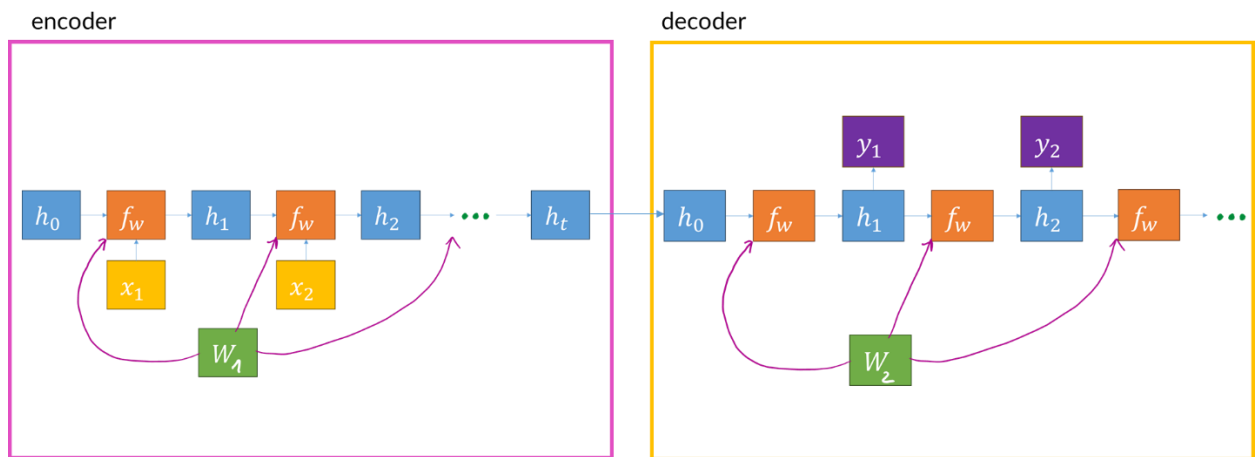


Cuando se realizan operaciones de tipo N a N, por ejemplo, una traducción, podemos también pensarlo como una primera red N a 1 más una segunda red 1 a N. En este caso, procederemos en dos etapas, lo que se conoce como codificador (encoder) y decodificador (decoder).

En el encoder, recibiremos la entrada de tamaño variable que podría ser una frase en español por ejemplo y luego resumiremos esa frase completa utilizando el estado oculto final de la red

del encoder. Por tanto, ahora tenemos el escenario N a 1 donde hemos sido capaces de resumir toda esa entrada de tamaño variable en un único vector.

A continuación, pasamos a la red del decoder de uno a N que tomará ese vector único y producirá una salida de tamaño variable que será por ejemplo la misma frase traducida al inglés.



Ejemplo RNN

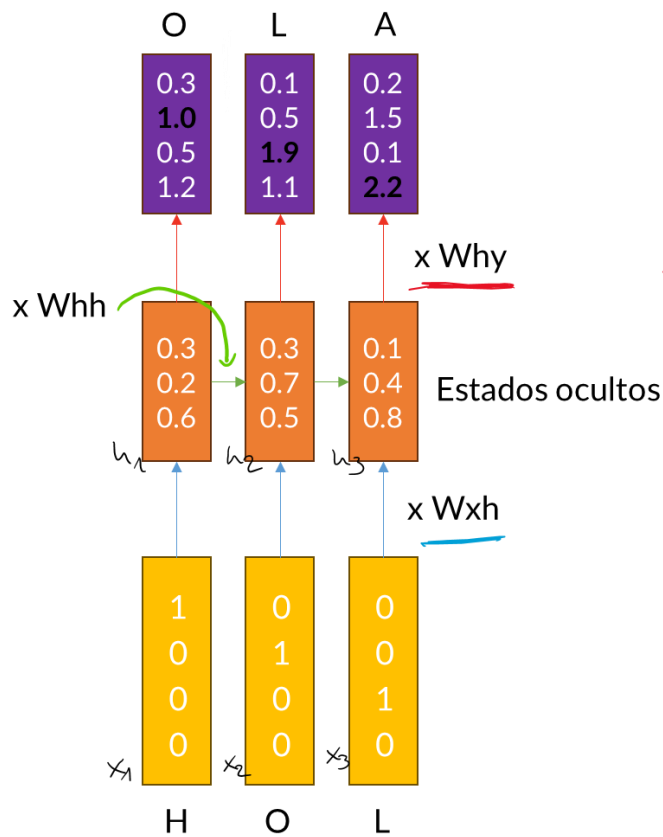
Como ejemplo de una de las cosas para las que se utilizan con frecuencia las redes recurrentes es para el problema de modelado del lenguaje donde queremos leer una secuencia de palabras que queremos que la red entienda cómo producir lenguaje natural. Esto puede suceder a nivel de carácter donde nuestro modelo genera cada letra de forma individual o a nivel palabra donde lo que se genera no es una letra sino una palabra, es decir, se generará palabra por palabra.

Si imaginamos este modelo del lenguaje carácter a carácter, vamos a ver un ejemplo sencillo donde se busca leer una serie de caracteres y necesitamos predecir el siguiente carácter.

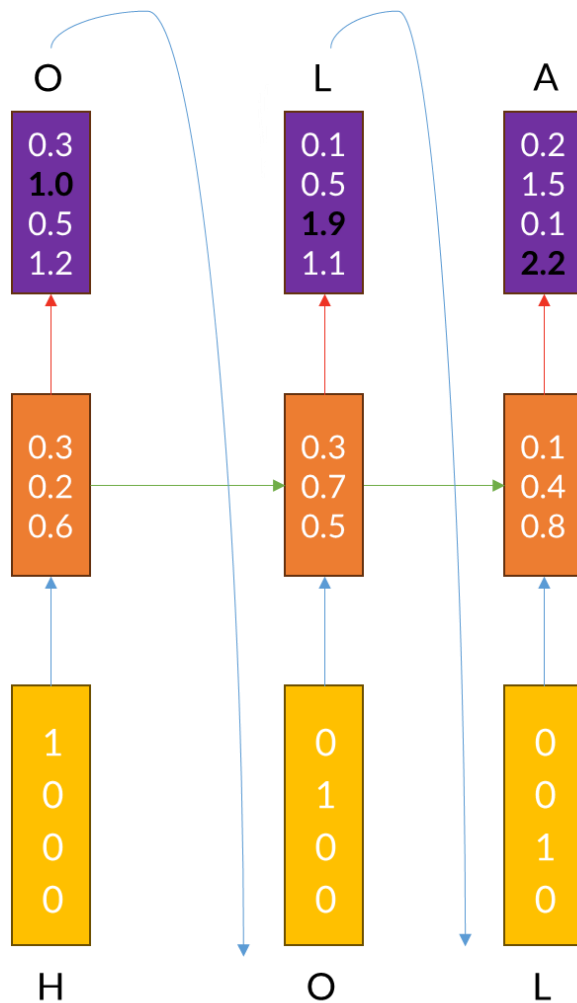
En este ejemplo, vamos a entrenar la palabra hola y nuestro alfabeto o vocabulario total se compone solo de 4 letras para simplificarlo h, o, l y a. En la realidad serían todas las letras del alfabeto.

Cuando entrenemos la red, le proporcionaremos los caracteres de la palabra hola en el orden correspondiente como entrada x . Cada letra se representa con un vector que tiene un 1 en la posición de la letra. El vector tiene 4 campos en total porque el vocabulario total consta de 4 letras. Cada una de las letras pasa por la operación que antes hemos visto que calcula h_t en base a x_t y h_{t-1} .

De esta forma, la red aprende qué letra va después de cuál, es decir, el concepto aprender implica actualizar los valores de los pesos de forma que cuando entra la H, sabe generar la O como output. El aprendizaje lo podemos conocer como truncated backpropagation through time.



Una vez la red está entrenada, cuando le pasamos la H, realiza los cálculos hasta que devuelve la O. La O se pasa como input de la siguiente iteración hasta que devuelve la L. Y se repite el mismo proceso hasta obtener la A.



11. Tema 9: Introducción a LSTM

Las Long Short-Term Memory (LSTM) son una variante de las Redes Neuronales Recurrentes (RNN) diseñadas para solucionar el problema del desvanecimiento del gradiente, que limita la capacidad de las RNN convencionales para aprender dependencias a largo plazo. Las LSTM incorporan celdas de memoria que permiten retener información a lo largo de secuencias largas, lo que las hace aptas para tareas que requieren procesamiento de datos secuenciales, como el procesamiento de lenguaje natural, la traducción automática y la predicción de series temporales.

¿Cómo funcionan las LSTM?

Las LSTM (Long Short-Term Memory) son una evolución de las RNN tradicionales. Incorporan una célula de memoria que conserva información relevante a lo largo de varios pasos de tiempo, evitando que se pierda o degrade. Esta célula está gestionada por puertas, que controlan el flujo de información, permitiendo a la red decidir qué información mantener y cuál descartar. A continuación, se analizarán los componentes clave que distinguen a las LSTM de las RNN

convencionales, destacando la célula de memoria y las puertas responsables de su funcionamiento.

Componentes clave de las LSTM

1. Célula de Memoria

Las LSTM (Long Short-Term Memory) utilizan una célula de memoria como su componente central, a diferencia de las RNN tradicionales que no pueden mantener información por períodos prolongados. Esta célula actúa como un "contenedor" de datos, permitiendo actualizarse, olvidarse o usarse según sea necesario. Su capacidad para conservar el contexto relevante a lo largo del tiempo permite que las LSTM manejen secuencias de datos extensas, adaptando su valor en función de la relevancia de la información, lo que mejora la toma de decisiones sobre qué retener y qué olvidar.

2. Puertas en LSTM

Las LSTM logran su capacidad de gestionar la información mediante el uso de **tres tipos de puertas**: la **puerta de olvido** (f_t), la **puerta de entrada** (i_t) y la **puerta de salida** (o_t). Estas puertas son mecanismos que controlan de manera precisa cómo fluye la información a través de la célula de memoria.

a) Puerta de Olvido (f_t)

La **puerta de olvido** es un componente crucial en la arquitectura de las LSTM, ya que decide cuánta información de la memoria previa debe descartarse. Este proceso de "olvido" es fundamental para asegurar que la red no acumule información irrelevante o que ya no es útil para la tarea en cuestión.

Matemáticamente, la puerta de olvido se define mediante una función sigmoide (σ), que toma como entrada el estado oculto anterior (h_{t-1}) y la entrada actual (x_t), y produce un valor entre 0 y 1. Este valor determina qué fracción de la memoria previa se mantendrá.

$$f_t = \sigma(w_f[h_{t-1}, x_t] + b_f)$$

Si el valor de f_t es cercano a 0, la red olvidará gran parte de la información almacenada en la célula de memoria anterior. Si el valor es cercano a 1, conservará esa información para utilizarla en pasos futuros. Esta puerta permite que la red sea flexible y selectiva, eliminando información innecesaria y enfocándose en lo que realmente importa en el contexto actual.

b) Puerta de Entrada (i_t)

La **puerta de entrada** es responsable de decidir qué nueva información debe almacenarse en la célula de memoria. Funciona en conjunto con una **nueva candidata de memoria** ($\tilde{c}_t$), que es calculada mediante una función tangente hiperbólica para generar una posible actualización de la memoria. La puerta de entrada usa esta nueva candidata para determinar qué parte de ella debe añadirse a la célula de memoria.

Al igual que la puerta de olvido, la puerta de entrada se define por una función sigmoide que genera un valor entre 0 y 1. Este valor controla la magnitud con la que la nueva información actualiza la memoria.

$$i_t = \sigma(\omega_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(w_c[h_{t-1}, x_t] + b_c)$$

La combinación de la puerta de entrada con la candidata de memoria permite a las LSTM incorporar información nueva de manera eficiente, ajustándose dinámicamente a las condiciones cambiantes de la secuencia.

c) Puerta de Salida (o_t)

La **puerta de salida** es la última de las tres puertas y es la encargada de determinar qué parte de la información contenida en la célula de memoria debe usarse para generar la **salida** en el paso de tiempo actual. Esto es importante porque no toda la información almacenada en la célula de memoria es relevante para la salida en un momento dado; la puerta de salida actúa como un filtro que decide qué información debe pasarse al estado oculto (h_t).

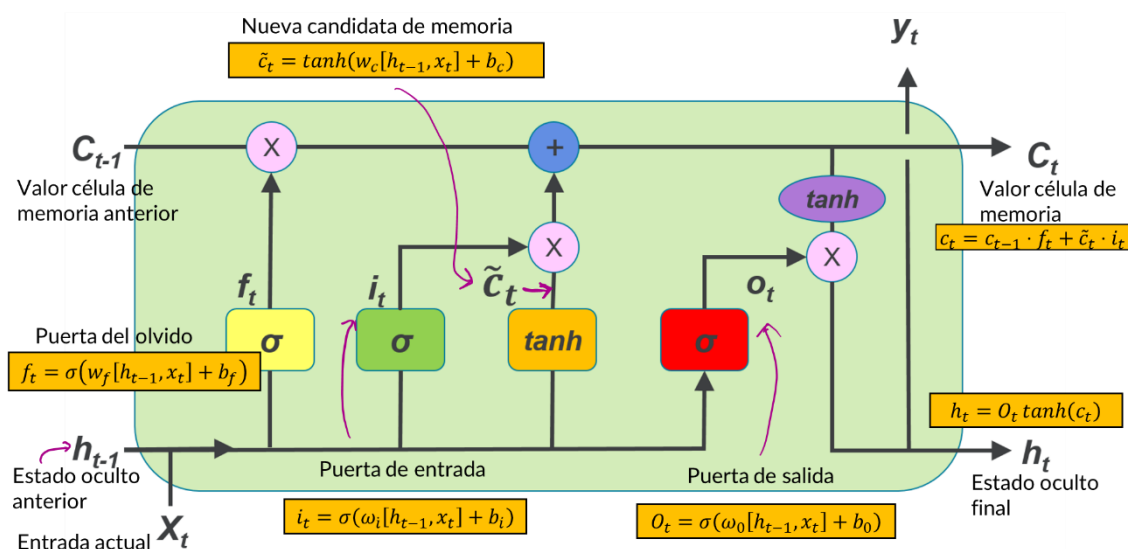
La puerta de salida también utiliza una función sigmoide para calcular un valor entre 0 y 1, que indica qué fracción de la memoria actual será utilizada en la salida.

$$O_t = \sigma(\omega_o[h_{t-1}, x_t] + b_o)$$

Una vez que se ha calculado el valor de o_t , este se multiplica por el valor actualizado de la célula de memoria pasada a través de una función tangente hiperbólica, lo que da lugar al estado oculto final h_t en el paso de tiempo actual:

$$h_t = O_t \tanh(c_t)$$

Este estado oculto es la salida que la red proporciona en el paso de tiempo actual, y es también lo que se pasa al siguiente paso de tiempo para que la red continúe procesando la secuencia.



Flujo de Información en las LSTM

Las LSTM (Long Short-Term Memory) mejoran las RNN estándar al gestionar eficientemente las dependencias a largo plazo en secuencias de datos. En cada paso de tiempo, las puertas de las LSTM determinan qué información mantener, olvidar o añadir, y qué memoria utilizar para generar la salida. Este proceso es clave en tareas como la traducción automática, donde se retiene y utiliza el contexto de palabras anteriores para garantizar una traducción precisa. Además, en predicciones como las del clima, las LSTM pueden recordar patrones pasados a largo plazo, mejorando la precisión de las predicciones. Su capacidad para manejar memoria y contexto a largo plazo las convierte en una herramienta crucial en aplicaciones de procesamiento de lenguaje natural y predicción de series temporales.

12. Tema 10: Comparación de LSTM con RNN estándar y GRU (Gated Recurrent Unit)

Comparación entre LSTM, RNN estándar y GRU

Las redes neuronales recurrentes (RNN) son fundamentales para procesar secuencias de datos como texto o audio, pero enfrentan limitaciones en la retención de información a largo plazo debido al desvanecimiento del gradiente. Para mejorar esto, se desarrollaron dos variantes: Long Short-Term Memory (LSTM) y Gated Recurrent Unit (GRU), que optimizan la capacidad de las RNN para manejar dependencias a largo plazo. Las RNN estándar son la base, donde cada neurona toma como entrada el dato actual y el estado oculto anterior. Las LSTM, introducidas en 1997, incorporan celdas de memoria para retener información durante períodos más largos, superando el problema del desvanecimiento del gradiente.

Ventajas de las LSTM

Las LSTM (Long Short-Term Memory) presentan varias ventajas, entre ellas:

1. Capacidad para manejar dependencias a largo plazo: Gracias a su célula de memoria y las puertas que controlan el flujo de información, las LSTM pueden recordar eventos distantes, superando a las RNN tradicionales en tareas que requieren contexto a largo plazo.
2. Flexibilidad: Las puertas de las LSTM permiten decidir dinámicamente qué información mantener o descartar, lo que las hace adaptables a diversos problemas secuenciales como el análisis de texto, la predicción de series temporales y la traducción automática.
3. Mitigación del desvanecimiento del gradiente: Las LSTM evitan que los gradientes se vuelvan muy pequeños durante la retropropagación, mejorando el aprendizaje en secuencias largas.

Gated Recurrent Unit (GRU)

Las Gated Recurrent Units (GRU), introducidas por Cho et al. en 2014, son una versión simplificada de las LSTM, diseñadas para abordar problemas como el desvanecimiento del gradiente y las dificultades en el manejo de dependencias a largo plazo en las RNN. A diferencia de las LSTM, que tienen tres puertas, las GRU cuentan con dos: la puerta de actualización, que combina las funciones de las puertas de olvido y entrada, y la puerta de reinicio, que determina cuánto de la memoria pasada se olvida. Esta estructura más sencilla hace que las GRU sean más

fáciles de entrenar y menos costosas computacionalmente, manteniendo un rendimiento similar al de las LSTM.

Comparación entre LSTM y GRU

Las LSTM y las GRU son dos arquitecturas populares de redes neuronales recurrentes, pero tienen diferencias clave:

1. Complejidad: Las LSTM son más complejas, con tres puertas y una célula de memoria dedicada, mientras que las GRU son más simples, con solo dos puertas y sin célula de memoria separada.
2. Rendimiento en secuencias largas: Las LSTM suelen ser más efectivas en secuencias largas o complejas debido a su célula de memoria, que permite un mayor control sobre la retención y el olvido de información.
3. Coste computacional: Las GRU son más rápidas y eficientes en términos computacionales debido a su arquitectura más simple, lo que las hace preferibles cuando el tiempo de entrenamiento es un factor importante.
4. Aplicaciones y rendimiento: Ambas arquitecturas ofrecen un rendimiento similar en tareas como predicción de series temporales o generación de texto. Las GRU pueden tener mejor rendimiento cuando se dispone de menos datos o las dependencias temporales no son complejas.
5. Capacidad de modelado: Las LSTM son más adecuadas para problemas que requieren modelar dependencias muy largas y un control preciso de la memoria, mientras que las GRU son más eficientes para problemas con menores necesidades de memoria a largo plazo.

Resumen Comparativo

Característica	RNN estándar	LSTM	GRU
Memoria a largo plazo	Limitada	Excelente	Muy buena
Problema del gradiente	Desvanecimiento	Mitigado	Mitigado
Complejidad	Baja	Alta	Media
Velocidad de entrenamiento	Rápida (en secuencias cortas)	Lenta (debido a más puertas)	Más rápida que LSTM
Número de puertas	N/A	3 (olvido, entrada, salida)	2 (actualización, reinicio)
Eficiencia computacional	Alta en secuencias cortas	Menos eficiente	Más eficiente que LSTM
Aplicaciones	Secuencias cortas	Secuencias largas y complejas	Secuencias de longitud media/larga

En esta comparativa, se concluye que las RNN estándar han quedado atrás frente a las LSTM y GRU, ya que no manejan eficientemente dependencias a largo plazo. Las LSTM, con su estructura compleja, son ideales para tareas con dependencias temporales largas, como la traducción automática, mientras que las GRU, más simples y eficientes, ofrecen un buen rendimiento con un menor costo computacional. La elección entre LSTM y GRU depende de la complejidad de la tarea y los recursos disponibles.

13. Tema 11: ¿Cuándo usar LSTM?

¿cuándo es más apropiado usar LSTM en lugar de otras arquitecturas de redes neuronales, como RNN estándar o GRU? A continuación, exploraremos los casos en los que las LSTM destacan y se convierten en la elección preferida para resolver problemas.

Dependencias temporales a largo plazo

Las redes neuronales LSTM (Long Short-Term Memory) destacan por su capacidad para recordar información relevante a lo largo de secuencias largas, lo cual es crucial en tareas donde el contexto pasado influye en decisiones futuras. Algunos ejemplos de dependencias a largo plazo incluyen:

- Traducción automática: Las LSTM permiten recordar palabras o frases anteriores para generar traducciones coherentes en oraciones complejas.
- Generación de texto: Mantienen el contexto y coherencia en textos largos, ayudando a preservar el estilo y la consistencia temática.
- Reconocimiento de voz: Recordar palabras previas es esencial para interpretar correctamente nuevas frases, lo que las LSTM facilitan.

En resumen, las LSTM son ideales para trabajar con secuencias largas donde se requiere recordar información de eventos pasados para generar resultados precisos.

Problemas con alta complejidad temporal

Las LSTM (Long Short-Term Memory) son útiles para problemas de series temporales con relaciones complejas debido a sus células de memoria y puertas (entrada, olvido y salida), que les permiten decidir qué información mantener y cuál descartar. Esto es especialmente útil en situaciones como:

- Predicción de series temporales: En áreas como finanzas, clima y energía, donde los datos están correlacionados a lo largo del tiempo, las LSTM pueden identificar patrones a largo plazo que las RNN tradicionales no captan.
- Control robótico o de sistemas: En aplicaciones como la robótica o el control industrial, las LSTM pueden modelar las relaciones entre acciones previas y el estado actual, manejando mejor la complejidad de estos sistemas.

En resumen, las LSTM son eficaces para gestionar dependencias temporales complejas a largo plazo.

Datos ruidosos o incompletos

Las LSTM (Long Short-Term Memory) son útiles para trabajar con datos secuenciales ruidosos o incompletos. Su capacidad para manejar inconsistencias y ruidos en los datos la hace más robusta que las redes neuronales estándar. En aplicaciones como el seguimiento de actividades físicas a través de sensores o la monitorización de signos vitales en medicina, los datos suelen estar contaminados o ser incompletos. Las LSTM pueden gestionar estas irregularidades, capturando tendencias importantes a lo largo del tiempo y reteniendo solo la información relevante, lo que las convierte en una opción eficaz en entornos con datos ruidosos o faltantes.

Tareas que requieren precisión en la memoria y contexto

Otra razón importante para elegir LSTM es cuando la precisión del contexto es crucial. En muchas aplicaciones, como el modelado de lenguaje, la traducción o la predicción de secuencias, el modelo debe ser extremadamente preciso en cómo maneja la memoria y el contexto a lo largo de la secuencia. Las LSTM proporcionan un control fino sobre qué información retener y cuál olvidar, lo que las hace ideales para estas tareas.

Ejemplos:

- **Modelado de lenguaje natural:** En tareas como el resumen automático de textos, el análisis de sentimiento, o la respuesta a preguntas basadas en texto, es fundamental que el modelo no solo retenga información relevante durante largos períodos de tiempo, sino que también lo haga con precisión. Si el modelo olvida o retiene la información incorrecta, los resultados pueden ser incoherentes o incorrectos.

- **Análisis de videos:** En problemas como la detección de acciones en videos, las LSTM pueden procesar secuencias de fotogramas, reteniendo información importante sobre el inicio de una acción mientras analizan el desarrollo completo del evento. Aquí, la capacidad de recordar detalles clave en el contexto de una secuencia larga es crucial.

En general, cuando la calidad del manejo de la memoria y el contexto afecta directamente el rendimiento de la tarea, las LSTM ofrecen una ventaja significativa sobre otras arquitecturas recurrentes más simples.

Cuando las RNN estándar no son suficientes

Finalmente, una situación común en la que se usan LSTM es cuando las **RNN estándar no logran el rendimiento esperado**. Dado que las RNN tradicionales tienen problemas para manejar secuencias largas debido al desvanecimiento del gradiente, si una RNN estándar tiene dificultades para aprender patrones temporales en los datos secuenciales, las LSTM suelen ser una mejora directa. Su capacidad para mitigar el desvanecimiento del gradiente les permite mantener el aprendizaje sobre secuencias más largas, lo que es esencial en muchas aplicaciones modernas.

Ejemplos:

- **Predicción de precios:** Si una RNN estándar no puede capturar correctamente la tendencia de los precios de las acciones o criptomonedas debido a la naturaleza a largo plazo de los ciclos del mercado, las LSTM pueden ser una mejor alternativa para captar estas fluctuaciones.
- **Reconocimiento de patrones en series temporales complejas:** En análisis de señales biomédicas, como electroencefalogramas (EEG) o electrocardiogramas (ECG), las dependencias a largo plazo en los datos pueden ser críticas para detectar anomalías. Si las RNN estándar no pueden capturar estas relaciones, las LSTM ofrecen una opción superior.

14. Tema 12: Limitaciones y Mejores Prácticas

Limitaciones de las LSTM

1. Complejidad y coste computacional

Las LSTM, aunque eficaces para manejar dependencias a largo plazo, son complejas y computacionalmente costosas debido a componentes adicionales como las puertas de entrada, salida y olvido, y las celdas de memoria. Esta estructura incrementa los parámetros a entrenar y, por ende, el tiempo y los recursos necesarios. En secuencias largas o con múltiples capas apiladas, los cálculos necesarios para cada puerta en cada paso de tiempo prolongan aún más el entrenamiento. Además, implementar LSTM en dispositivos con recursos limitados es desafiante debido a sus altos requisitos computacionales.

2. Problemas con secuencias extremadamente largas

Las LSTM superan a las RNN estándar en retención de información a lo largo del tiempo, pero aún presentan dificultades con secuencias muy largas. En estos casos, puede ser complicado para el modelo captar datos críticos de los primeros pasos, pues, aunque las celdas de memoria de las LSTM reducen el problema del desvanecimiento del gradiente, la información distante pierde relevancia con el tiempo. Este desafío es evidente en tareas como la traducción de textos extensos o la predicción en series temporales, donde es crucial recordar dependencias al principio de la secuencia para hacer predicciones precisas al final.

3. Riesgo de sobreajuste

Las LSTM, como otros modelos de redes neuronales profundas, tienden a sobreajustarse, especialmente con conjuntos de datos pequeños. Esto ocurre porque pueden memorizar detalles específicos de las secuencias de entrenamiento que no se generalizan bien a datos nuevos. Este problema se agrava cuando el modelo tiene demasiados parámetros o cuando los datos de entrada son ruidosos.

Mejores prácticas para usar LSTM

Para maximizar el rendimiento de las LSTM (Long Short-Term Memory) y mitigar sus limitaciones, es clave implementar varias buenas prácticas:

1. Normalización de datos: Normalizar o estandarizar los datos de entrada, especialmente en secuencias temporales, facilita el aprendizaje de patrones al mantener equilibrados los gradientes, evitando problemas de aprendizaje como gradientes explosivos o desvanecientes.
2. Regularización para evitar el sobreajuste: Para manejar el alto número de parámetros de las LSTM, se recomienda el uso de regularización, como Dropout en las conexiones recurrentes y capas de salida, además de L2 regularization, lo cual ayuda a evitar que el modelo se ajuste excesivamente a los datos de entrenamiento.
3. LSTM bidireccionales: Las capas bidireccionales permiten al modelo considerar tanto el contexto anterior como posterior en una secuencia, lo cual es útil en tareas donde el contexto completo mejora la precisión, como en el procesamiento de lenguaje natural.
4. Modelos apilados: Agregar múltiples capas de LSTM permite al modelo capturar representaciones más complejas y relaciones jerárquicas. Sin embargo, esto aumenta el riesgo de sobreajuste, por lo que se deben equilibrar las capas apiladas con regularización.
5. Ajuste de hiperparámetros: Ajustar el número de unidades LSTM, la tasa de aprendizaje y el tamaño del lote permite optimizar el modelo para el problema específico. La experimentación y la validación cruzada ayudan a encontrar la configuración ideal.

Estas prácticas contribuyen a mejorar la eficiencia y generalización de las LSTM, haciendo que el modelo sea más robusto y preciso en tareas de secuencias complejas.

15. Bibliografía

Deep Learning: A Practitioner's Approach. Josh Patterson, Adam Gibson. O'Reilly

Lectura recomendada: <https://www.deeplearningbook.org/>

- Lecturas recomendadas:

- "Deep Learning" de Ian Goodfellow (Capítulo 10).
- Artículo original de LSTM: "Long Short-Term Memory" de Sepp Hochreiter y Jürgen Schmidhuber (1997).

16. Conclusiones

El Deep Learning ha revolucionado múltiples sectores gracias a su capacidad de aprender y extraer patrones de grandes volúmenes de datos, lo que ha permitido avances significativos en la automatización y la toma de decisiones. Su impacto es evidente en áreas como la visión por computadora, el procesamiento del lenguaje natural y la atención médica, donde permite soluciones más inteligentes y eficientes. A medida que esta tecnología evoluciona, se espera que su rol en el desarrollo de aplicaciones innovadoras y servicios sofisticados se vuelva aún más esencial para resolver problemas complejos.

Las redes de Long Short-Term Memory (LSTM) representan una mejora clave dentro de las Redes Neuronales Recurrentes (RNN), abordando la limitación del desvanecimiento del gradiente que enfrentan las RNN estándar al procesar secuencias largas. Las LSTM incluyen una célula de memoria y un conjunto de "puertas" que controlan el flujo de información, permitiendo retener datos relevantes durante periodos prolongados. Esto las hace ideales para tareas secuenciales como el procesamiento del lenguaje natural y la predicción de series temporales, donde la capacidad para recordar información a largo plazo es fundamental.

WELCOME
TO
UAX

UAX

Universidad
Alfonso X el Sabio

GRACIAS

UAX.COM